数据的表示与计算

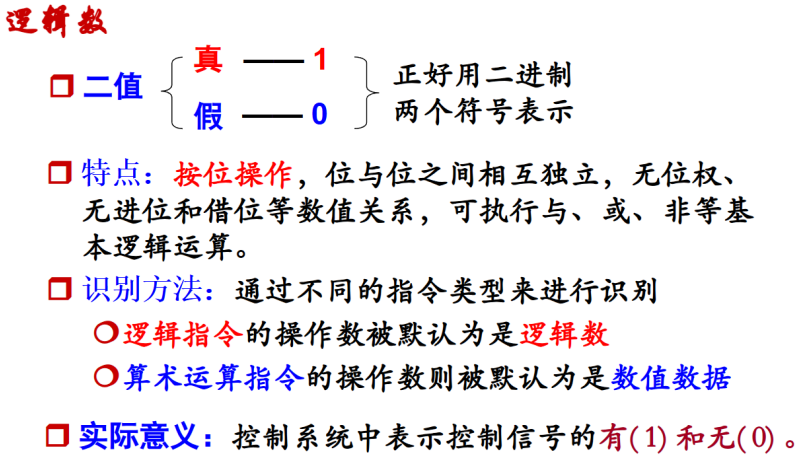
数据的表示与计算

- 计算机中表示信息的基本方法
- 定点数的表示
  - 真值与机器数
  - 原码、反码、补码
- 定点运算
  - 定点乘法
  - 定点除法
  - 阵列乘除法器
  - 十进制运算
  - 逻辑运算
- 定点运算器的实现
  - 加法器的进位技术
  - 算术逻辑单元ALU
  - 定点运算器的基本结构
- 浮点数的表示与运算
- 考点

计算机中表示信息的基本方法

信息表示：基二码（二进制编码）:易于物理实现，运算的简易性，很高的经济性，适合逻辑代数的应用

“数据”泛指所有需要加工处理的信息



字符的表示

字符——字母、数字、专用符号等西文信息，是人与计算机交互的重要媒介。

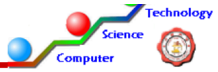
机内表示的关键：二进制编码，每个字符都有唯一的编码值，作为识别的依据。

❍字符交换码标准

ASCII码——美国标准信息交换码（American Standard Code for Information Interchange），当前国际通用。

✧ ASCII码字符集：7位二进制编码表示一个特定的字符，总共定义了128种字符， 95种显示字符，33种控制字符。

ASCII码的机内表示



ASCII码在计算机中用一个字节来进行表示，ASCII码只占用了字节的低7位，字节的最高位取值需要确定。

处理方法：

(1) 恒为“0”，可利用这一位作为ASCII码识别标志

(2) 奇偶校验位，根据奇偶校验技术的需要取值(1/0)

(3) 扩展ASCII码方案，最高位用作字符编码(1/0)

✧ASCII码位数扩展到8位

✧字符集扩大到256种字符

字符串：向量表示，存放在主存连续多个字节单元中，每个字节保存串中一个字符的ASCII码。（小端/大端）

汉字：数字编码、字音编码、字形编码、形音编码，机内码用国标码，两字节最高位1，其余7位国标码。描述方法：点阵/轮廓

十进制数：BCD码->8421，余3，2421，gray

十进制数串：（1）字符串形式：前分隔数字串/后嵌入数字串（针对符号）

（2）压缩的十进制数串：

压缩的十进制数串形式

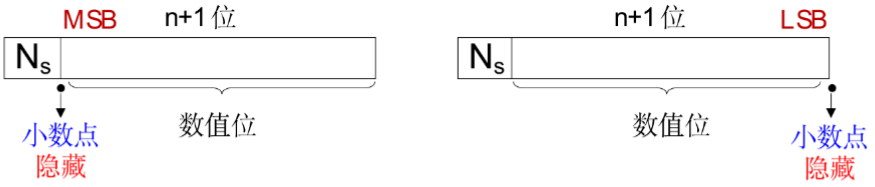
- 串中每位十进制数的ASCII码高4位压缩掉，只用其低4位，相当于每位十进制数用8421码表示，占用半个字节。
- 符号位：选用8421码不用的6种冗余编码表示，并放在最低数位之后。通常用1100(CH)表示正号，1101(DH)表示负号。
- 规定数值位加符号位之和必须为偶数，若串中总位数连同符号位为奇数时，在最高位前补0。
- 表征参数：串首地址和位长（不含符号位）。位长可变，常为0~31，甚至更长。位长为0的数其值也为0。
- 压缩的十进制数串比字符串表示方式节省一倍左右的存储空间，又便于直接完成十进制数的算术运算，是较为理想的表示方法。

定点数的表示

真值与机器数

无符号数/带符号数（前置0/1）

小数的**定点表示法**：指小数点位置在计算机中式事先约定好且固定不变的。分为**定点整数表示法**（小数点在最低数值位之后）和**定点小数表示法**（小数点位置在符号位之后）



a）定点小数　　b）定点整数

定点表示法的特点：表示方法简单，但表示范围较小，（纯整数、纯小数）。原始数据需乘上一定的比例因子，变为整数或小数形式，定点计算机内部才能接受。对于非常大或非常小的数据，合适的比例因子不好找。

真值：用一般书写形式表示的数

机器数：计算机内数据的形式（机器数编码方案：原码、反码、补码、移码）

计算机内运算的特点：模运算。模溢出：模运算系统中编码表示范围有上限，运算时一旦达到此值即自动丢掉

原码、反码、补码

补码：正不变；负数高位变反，低位不变。补码可比原码多表示一个数，下限为`-1.000...`，其补码为`1.000 0000`，`1`既表示负号也表示数值。计算方法：拿2(10.000...)减；取反+1

变形补码：双符号位，正`00`，负`11`。变为模4补码，表示范围扩大了一倍



- 整数原、补、反码与小数表示基本相同，仅小数点位置不同
- 符号位按0正1负设置，原码、补码、反码均相同；
- 正数 原、补、反码均相同，符号位为 0，数值位同数的真值
- 原码和反码零的表示不唯一，补码零的表示唯一；
- 负数的原码、补码、反码表示均不同，符号位为 1，
  - 数值位：原码为数的绝对值；
  - 反码为绝对值按位取反；
  - 补码高位部分同反码，低位部分同原码；
- 补码数值位与原、反码转换规则双向适用，符号位不需转换
- 原、反码定义域相同，表示范围零对称。补码正、负定义域不对称，正数表示范围同原码，负数较正数能多表示1个数
- 补码和反码的符号位和数值位一起参加运算；原码符号位不能参加运算，必须单独进行处理。

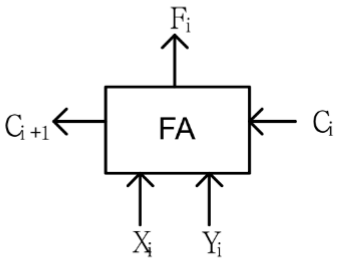
定点表示法特点：1. 表示方法简单，运算简单，结构简单，硬件代价低；2. 表示范围较小，运算结果容易超出其表示范围，导致溢出；3. 只能表示纯小数或纯整数，需要选择合适比例因子

定点运算

运算部件：全加器；n位加法器：串行/并行

一位二进制加法单元

全加器的逻辑符号



全加器真值表

输 入			输 出	
X_i	Y_i	C_i	F_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

全加器的典型逻辑表达式

$$F_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_iY_i + (X_i \oplus Y_i)C_i, \ (\text{或} C_{i+1} = X_iY_i + (X_i + Y_i)C_i)$$

补码的加减运算（定点小数）：

补码加减运算基本公式

$$[X \pm Y]_{补} = [X]_{补} \pm [Y]_{补} \ (\text{mod } 2)$$

即：$[X+Y]_{补} = [X]_{补} + [Y]_{补} \ (\text{mod } 2)$

$$[X-Y]_{补} = [X]_{补} + [-Y]_{补} \ (\text{mod } 2)$$

求$[-Y]_{补}$，可通过对$[Y]_{补}$逐位取反,再在最低位加1完成。

例：已知真值X、Y，试用补码加减算法求X±Y=?

X=0.1011011，Y=-0.0010010

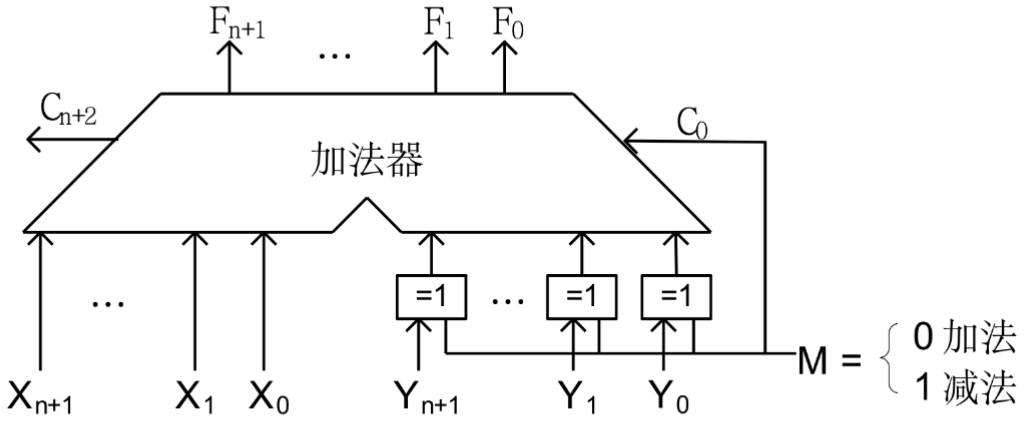
解：$[X]_{补}=X=0.1011011$； $[Y]_{补}=1.1101110$； $[-Y]_{补}=0.0010010$

$[X+Y]_{补}=0.1011011+1.1101110$

$=10.1001001=0.1001001$ (mod 2) （模溢出，自动丢2）

$[X-Y]_{补}=0.1011011+0.0010010=0.1101101$

X+Y=[X+Y]补=0.1001001； X-Y=0.1101101



特点：

- 在加法器的一个输入端添加一组异或门实现求补运算；
- 在异或门的输入端设置加/减控制信号M进行运算选择。

溢出（overflow）：运算结果超出数据的表示范围，分为正溢出和负溢出；判断方法：正+正=负/负+负=正；双符号位01或10，高位为正确符号。

例：(1) X=-0.1101100，Y=-0.1011011，求X+Y

(2) X=0.1101100，Y=-0.1011011，求X-Y

解：(1) $[X]_{补}$= 1.0010100；$[Y]_{补}$=1. 0100101；$[-Y]_{补}$=0.1011011

$[X+Y]_{补}$=1.0010100+1. 0100101=10.0111001=0.0111001（mod 2）

分析：① 模溢出，自动丢2（不影响结果正确性）；

② 负加负结果应为负，但现在结果符号位为0，出错（负溢出）

(2) $[X]_{补}$= 0.1101100；$[Y]_{补}$=1. 0100101；$[-Y]_{补}$=0.1011011

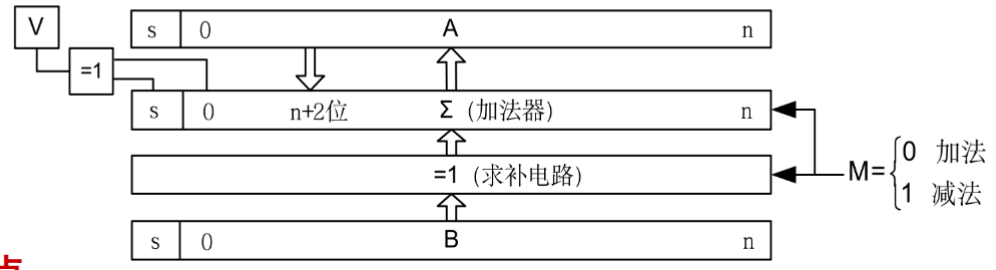
$[X-Y]_{补}$=0.1101100＋0.1011011=1.1000111

分析：正数减去负数结果应为正，但现在结果符号位为1，出错（正溢出）

溢出的后果：结果的最高数值位侵入符号位，使符号位遭到破坏。在计算机中溢出是作为出错处理的。

模四补码运算逻辑：

模4补码加减运算器逻辑框图



特点：

- 数值位为n位，运算器的总位数应达到n+2位；
- 补码加减运算器主要由一个加/减法器和两个寄存器组成；
- 寄存器A为累加器，用来存放运算结果，下一次运算可继续使用；
- 加上溢出判别、操作控制等辅助逻辑电路，完善加减运算功能；
- 整数补码加减与定点小数补码加减法运算原理一样，硬件线路也完全相同，只是小数点的默认位置在操作数的最低位（LSB）之后

移位：

- 逻辑移位：不考虑数据符号位含义的移位操作，对无符号数进行移位
- 算数移位：对带符号数进行的移位，移位过程中保持符号位不变

原码、补码、反码的算术移位空位填补规则：

	码　制	右移填补代码	左移填补代码
正数	原码、补码、反码	0	0
负数	原　码	0	0
	补　码	1	0
	反　码	1	1

- 循环移位：将机器字首尾相接进行的移位，高位补低，低位补高。带进位的循环移位：将CF加入移位循环回路，将移出位在粮寄存器间传递

移位后对值的影响

正数：左移时最高数位丢1，结果溢出出错；
右移时最低数位丢1，影响精度。

负数：原码左移时，高位丢1，结果溢出出错；
右移时，低位丢1，影响精度；
反码左移时，高位丢0，结果溢出出错；
右移时，低位丢0，影响精度；
补码左移时，高位丢0，结果溢出出错；
右移时，低位丢1，影响精度；

算术右移误差的舍入：在舍去多余位时，按照一定规则对保留的数值位进行调整（硬件：保护位寄存器）

方法：

(1) 截断法：无条件丢弃，单向误差，累积误差

(2) 末位恒置1法：在舍去结果最低位之后数值同时将机器数末位置1，双向误差，精度更高。当机器数最低位=1或移出的位有1则置1

(3) 0舍1入法：用移出部分的最高位作判断标志，若为0则舍，直接截断；若为1则入，丢掉移出位后对数值保留部分最低位+1。双向误差，无累积误差，精度最高。若补码，最高位为1，其余各位为0，也舍去。原码舍绝对值减小，入绝对值增大；补码相反。

例：设X_1=0.0111000010，Y_1=-0.0111000010；
X_2=0.0111001100，Y_2=-0.0111001100；

分别用原码和补码表示，如果只要求8位字长，请分别采用截断法、恒置1法和0舍1入法对每一个操作数进行舍入，并对舍入结果进行比较。

解：先将真值X_1、X_2、Y_1、Y_2表示成机器码形式，再进行舍入，为方便比较，舍入结果用表格列出（见下页）。
注意相同下标的X_i、Y_i互为相反数，LSB*则表示误差方向是相对于最低有效位LSB的绝对值而言。

舍入前（11位）	舍入后（8位）	丢掉位	结果真值	误差分析
$[X_1]_{原}=[X_1]_{补}=X_1$ =0.0111000010	截断=0舍1入 =0.0111000 (舍) 恒置1=0.0111001 (入)	010	0.0111 0.0111001	-1/4LSB* +3/4LSB*
$[Y_1]_{原}$=1.0111000010	截断=0舍1入 =1.0111000 (舍) 恒置1=1.0111001 (入)	010	-0.011 1 -0.0111001	-1/4LSB* +3/4LSB*
$[Y_1]_{补}$=1.1000111110	截断=恒置1 =1.1000111 (入) 0舍1入=1.1001000 (舍)	110	-0.0111001 -0.0111	+3/4LSB* -1/4LSB*
$[X_2]_{原}=[X_2]_{补}=X_2$ =0.0111001100	截断=恒置1 =0.0111001 (舍) 0舍1入=0.0111010 (入)	100	0.0111001 0.011101	-1/2LSB* +1/2LSB*
$[Y_2]_{原}$=1.0111001100	截断=恒置1 =1.0111001 (舍) 0舍1入=1.0111010 (入)	100	-0.0111001 -0.011101	-1/2LSB* +1/2LSB*
$[Y_2]_{补}$=1.1000110100	截断=0舍1入 =1.1000110 (入) 恒置1=1. 1000111 (舍)	100	-0.011101 -0.0111001	+1/2LSB* -1/2LSB*

误差计算：**看"丢掉位"得到 f，然后：**

- 截断：$e = -f \cdot LSB*$
- 恒置1：$e = (1-f) \cdot LSB*$
- 0舍1入：$e = (\delta - f) \cdot LSB*$

其中 $f = 0.(\text{丢掉位})_2$, δ 就是丢掉位首位（0 或 1）。

移位硬件：移位寄存器/组合逻辑

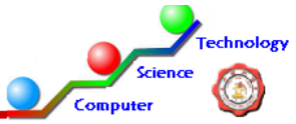
组合逻辑的移位电路是一个多路选择器，可实现多位并行移位

桶形移位器是一种并行度极高的快速移位器，可在数据通路宽度范围内将一个数据移动任意位。基于组合逻辑多路选择器。

定点乘法

1、原码一位乘法

原码一位乘法运算规则

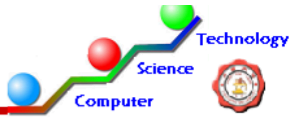


- 参加运算的两个数均为原码，两数符号位不参加运算，取其绝对值做无符号数乘法；
- 乘积的符号位单独处理，由两数符号“异或”产生；
- 设部分积初值为0。为防止运算过程中产生暂时性溢出，部分积采用单符号位参加运算。n位数相乘，部分积连同符号位应取n+1位；
- 运算前先检测0，只要两数有任意一个为0，则乘积为0，不再进行运算；
- 部分积运算：用乘数的最低位作为判断位，若为1，加被乘数；若为0，不加（相当于加0）；运算后部分积逻辑右移一位。
- 当乘数数值部分为n位时，共作n次“加法”和“右移”，最后得到2n位的乘积绝对值。

方法: 1. 写出原码； 2.写出绝对值X* /Y* ； 3. 写出X_s, Y_s(X与Y符号位)，做异或得到结果符号位；4. 列竖式计算$X^* \times Y^*$

计算：部分积初始为0，乘数用Y*，从小数点后开始。部分积和乘数一起右移一位，舍掉最后一位，看乘数最后一位现在是什么。是0就继续右移，是1则给部分积+X*。反复操作，直到乘数部分的小数点移到最后停止。将部分积与乘数拼起来得到结果。最后把符号拼上。

原码一位乘法运算实例



例： X = 0.1101　　Y = - 0.1011

$[X]_{原}$ = 0.1101　　$[Y]_{原}$ = 1.1011

则 X* = 0.1101　　Y* = 0.1011

手工运算过程

```
   0.1101
 × 0.1011
 ------
   1101
  1101
 0000
+1101
 ------
0.10001111
```

$[X \times Y]_{原}$ = 1.1000 1111

X×Y = -0.1000 1111

部分积	乘数
0.0000	0.1011
+0.1101	
0.1101	
→ 0.0110	10.101
+0.1101	
1.0011	
→ 0.1001	110.10
→ 0.0100	1110.1
+0.1101	
1.0001	
→ 0.1000	1111 0.

计算机内运算的描述方法

整数原码一位乘法：注意用逗号

整数原码一位乘法

- 原码一位乘算法规则同样适用于整数，只要把例题中的小数点“.”改为逗号“,”即可。
- 注意整数小数点的隐含位置和定点小数不一样，故乘积最终要向右对齐小数点，这是两种定点数据格式运算时的主要差别。

- 例：X＝1101，Y＝- 1011，用原码一位乘法求X×Y=?
- 解：$[X]_{原}$= 0,1101，$[Y]_{原}$=1,1011

  X^*= 0,1101，　Y^*=1011

  $P_s=X_s\oplus Y_s=0\oplus 1=1$；$P^*$=0, 1000 1111

  $[P]_{原}=[X\times Y]_{原}$=1, 1000 1111

  X×Y= - (0,1101×1011) = - 1000 1111

原码乘法运算器主要由一个加法器和三个寄存器组成。

2、补码一位乘比较法：

补码一位乘（比较法）算法规则

(1) X、Y均用补码表示，且符号位都参加运算，乘积的符号位由运算过程自动产生；

(2) 设部分积初值为0，部分积采用双符号位运算，但乘数只需要一位符号位参加运算；

(3) 乘数最低位之后增加一位附加位Y_{n+1}，且初始令Y_{n+1}=0；

(4) 运算前检测0，只要X、Y有任意一个为0，则乘积为0，不再运算；

(5) 用乘数的最低两位Y_nY_{n+1}作为判断位，其比较值(Y_{n+1}-Y_n)始终决定了每次应执行的操作。

- 部分积运算规则如下：

  Y_nY_{n+1}= 00 时，$[Z_i]_{补}=[Z_{i-1}]_{补}+0$，　算术右移1位；

  Y_nY_{n+1}= 01 时，$[Z_i]_{补}=[Z_{i-1}]_{补}+[X]_{补}$，　算术右移1位；

  Y_nY_{n+1}= 10 时，$[Z_i]_{补}=[Z_{i-1}]_{补}+[-X]_{补}$，　算术右移1位；

  Y_nY_{n+1}= 11 时，$[Z_i]_{补}=[Z_{i-1}]_{补}+0$，　算术右移1位；

(6) 重复n+1次比较和运算，但只右移n次，最后一次只运算不移位。

(7) 运算完成后，为避免误差，乘数的最低两位Y_nY_{n+1}清0。

方法：1. 写出X、Y补码；　2. 写出-X补码；　3.计算X*Y补码

计算方法：部分积初始为0，符号位两位（00.000...）。乘数用$Y_{补}$，乘数最后一位为Y_n，新加一位Y_{n+1}为0。算术右移（保留符号位），看Y_nY_{n+1}结果：

- 为00，直接右移

- 为01，部分积+$[X]_{补}$
- 为10，部分积+$[-X]_{补}$，
- 为11，直接右移

当小数点到Y_n后时，再运算一次，不右移，删掉Y_{n+1}，Y_n清零，将部分积和乘数部分拼起来得到结果。

补码一位乘（比较法）运算实例

已知：X = 0.1101
Y = - 0.1011
求：X×Y =?
解：$[X]_{补}$ = 0.1101
$[Y]_{补}$ = 1.0101
$[-X]_{补}$ = 1.0011
$[X \times Y]_{补}$ = 11.0111000100
X×Y = -0.10001111

部分积	乘数 y_n y_{n+1}
0 0.0 0 0 0	1.0 1 0 1 0
+1 1.0 0 1 1	
1 1.0 0 1 1	
→1 1.1 0 0 1	1 1.0 1 0 1
+0 0.1 1 0 1	
0 0.0 1 1 0	
→0 0.0 0 1 1	0 1 1.0 1 0
+1 1.0 0 1 1	
1 1.0 1 1 0	
→1 1.1 0 1 1	0 0 1 1.0 1
+0 0.1 1 0 1	
0 0.1 0 0 0	
→0 0.0 1 0 0	0 0 0 1 1. 0
+1 1.0 0 1 1	
1 1.0 1 1 1	0 0 0 1 0 0
	清0

整数补码一位乘：整数补码乘积右移后全看成符号位。

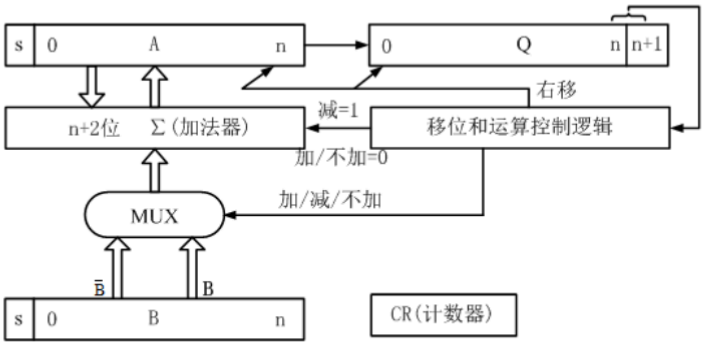
整数补码一位乘（比较法）算法

- 补码一位乘比较法规则同样适用于整数，也需要把例题中的小数点“.”改为逗号“，”，关注的焦点仍是小数点的隐含位置不同于小数。
- 为了硬件实现方便，可将小数点的位置约定在乘数的附加位之后，这意味着乘数在运算前被乘了2。为了保证双倍字长整数乘积的小数点在其最低位（LSB）的右边对齐，最后一步运算后，结果需要算术右移2位。
- 根据补码的符号扩展特性，整数补码乘积高位与符号位取值相同的部分，应该全部看成扩展的符号，而不是有效的数值位。

实现：

补码一位乘法运算的实现

补码一位乘比较法运算器



特点：与原码一位乘法运算器类似，主要区别为：

- 运算器的位数为n+2位（双符号位运算）；
- 乘数单符号位参加运算，Q寄存器最低位用作附加位Y_{n+1}；
- 核心部件为加/减法器， 由n+2个与或控制门电路控制输入；
- Q寄存器的最低两位Q_nQ_{n+1}控制部分积作何种运算；

3、补码两位乘比较法

补码两位乘比较法算法规则

(1) X、Y均用补码表示，且符号位都参加运算，乘积的符号位由运算过程自动产生；

(2) 部分积采用三位符号位运算，初值为0；

(3) 乘数最低位之后增加一位附加位Y_{n+1}，初始令$Y_{n+1}=0$；

(4) 运算前检测0：只要X、Y有任意一个为0，则乘积为0，不再进行运算；

(5)设乘数数值部分为n位，

当n为奇数时，乘数设1位符号位，做(n+1)/2次运算和移位，最后一步右移1位；

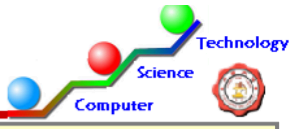
当n为偶数时，乘数设2位符号位，做(n/2)+1次运算，n/2次移位，最后一步不移位。

(6) 运算完成后，为避免误差，乘数的最低三位$Y_{n-1}Y_nY_{n+1}$清0。

(7)用乘数最低三位作判断位，每求一次部分积右移两位，部分积运算规则如下：

$y_{n-1}\ y_n\ y_{n+1}$= 000 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+0$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 001 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+[X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 010 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+[X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 011 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+2[X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 100 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+2[-X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 101 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+[-X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 110 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+[-X]_{补}$，　算术右移2位；
$y_{n-1}\ y_n\ y_{n+1}$= 111 时，$[Z_{i+2}]_{补}=[Z_i]_{补}+0$，　算术右移2位；

补码两位乘法运算实例



部分积	乘数 y_n y_{n+1}
0 0. 0 0 0 0	1.0 1 0 1　0
+1 1. 0 0 1 1	
1 1. 0 0 1 1	
1→1 1. 1 0 0 1	1 1.0 1 0　1
+0 0. 1 1 0 1	
1 0 0. 0 1 1 0	
1→0 0. 0 0 1 1	0 1 1.0 1　0
+1 1. 0 0 1 1	
1 1. 0 1 1 0	
1→1 1. 1 0 1 1	0 0 1 1.0　1
+0 0. 1 1 0 1	
1 0 0. 1 0 0 0	
1→0 0. 0 1 0 0	0 0 0 1 1.　0
+1 1. 0 0 1 1	
1 1. 0 1 1 1	0 0 0 1 0　0
	清0

部分积	乘 数 y_n y_{n+1}
0 0 0. 0 0 0 0	1 1.0 1 0 1　0
+ 0 0 0. 1 1 0 1	
0 0 0. 1 1 0 1	
2→0 0 0. 0 0 1 1	0 1 1 1.0 1　0
+ 0 0 0. 1 1 0 1	
0 0 1. 0 0 0 0	
2→0 0 0. 0 1 0 0	0 0 0 1 1 1.　0
+ 1 1 1. 0 0 1 1	
1 1 1. 0 1 1 1	0 0 0 1 0 0　0
	清0

已知：$[X]_{补}$ = 0.1101
　　$[Y]_{补}$ = 1.0101
则 $[-X]_{补}$ = 1.0011
　　$[X\times Y]_{补}$ =111.01110001000

比较乘法运算总结

乘法类型	符号位			累加次数	移位		
	参加运算	部分积	乘数		方向	次数	每次位数
补码一位乘法	是	2	1	$n+1$	右	n	1
补码两位乘法	是	3	2(n为偶数)	$\frac{n}{2}+1$	右	$\frac{n}{2}$	2
			1(n 为奇数)	$\frac{n+1}{2}$	右	$\frac{n+1}{2}$	2(最后一次移1位)

注：n为乘数的数值部分的位数

定点除法

1、原码恢复余数除法

原码恢复余数除法

例如：

X = -0.1011

Y = -0.1101

求：X÷Y=?

解：

$[X]_{原}$= 1.1011

$[Y]_{原}$= 1.1101

X^* = 0.1011

Y^* = 0.1101

$[-Y^*]_{补}$ = 1.0011

$[X/Y]_{原}$=0.1101

$R^*=0.0111\times2^{-4}$

被除数（余数）	商	说　明
0.1011	0.0000	
+1.0011		+$[-Y^*]_{补}$（减除数）
1.1110		余数< 0, 商上0
+0.1101		恢复余数 +Y^*
0.1011		
←1.0110	0.0000	左移一位
+1.0011		+$[-Y^*]_{补}$
0.1001		余数> 0, 商上1
←1.0010	0.0001	左移一位
+1.0011		+$[-Y^*]_{补}$
0.0101		余数> 0, 商上1
←0.1010	0.0011	左移一位
+1.0011		+$[-Y^*]_{补}$
1.1101		余数< 0, 商上0
+0.1101		恢复余数 +Y^*
0.1010		
←1.0100	0.0110	左移一位
+1.0011		+$[-Y^*]_{补}$
0.0111	←0.1101	余数> 0, 商上1

2、原码加减交替除法

原码加减交替除法算法规则

- 参加运算的数均为原码，两数符号位不参加运算，取其绝对值做无符号数除法；
- 商的符号位单独处理，由两数符号“异或”产生；
- 除法运算前检测0：只要X、Y有任意一个为0，不进行实际运算；

  若X^*为0，则商为0；若Y^*为0，按非法除数处理；
- 运算前应满足条件：$X^*<Y^*$，否则按溢出处理；
- 设商的初值为0（被除数单倍字长）或为被除数低位部分（被除数双倍字长）；

  部分余数的初值为被除数（被除数单倍字长）或为被除数高位部分（被除数双倍字长）。
- 部分余数采用单符号位，且符号位初值为0（取X^*）；

(7) 部分余数运算：

试减 $X^*+[-Y^*]_{补}$

{ 余数为正，商上1，余数和商左移一位，减除数；
  余数为负，商上0，余数和商左移一位，加除数；}

重复n步

(8) 第n+1步：若余数为正，商上1，商左移一位，余数不移位，运算结束；

若余数为负，商上0，商左移一位，但当结果需要余数时，需恢复余数（余数不移位直接加除数）。

$R^* = R_n \times 2^{-n}$。

方法：准备阶段同乘法，要算出$[-Y^*]_{补}$。

除法竖式：左为被除数/余数，右为商，设置商位数与被除数一致。先试减（被除数$+[-Y^*]_{补}$），结果符号位若为1，商为0.，余数$+Y^*$; 若为0，商为1.，余数$+[-Y^*]_{补}$。当商位数与被除数相同时（到达最后一步），若商最后一位为0，余数需要再$+Y^*$, 若商最后一位为1，则运算结束。

原码加减交替除法举例

X = -0.1011

Y = - 0.1101

求： X÷Y=?

解：$[X]_{原}$= 1.1011

$[Y]_{原}$= 1.1101

X^* = 0.1011

Y^* = 0.1101

$[-Y^*]_{补}$ = 1.0011

$[X÷Y]_{原}$= 0.1101

X÷Y= 0.1101

$R^*=0.0111\times2^{-4}$

被除数（余数）	商	说　明
0.1011	0.0000	
+ 1.0011		+ $[-Y^*]_{补}$ (减除数)
1.1011		余数< 0, 商上0
←1.0110	0.0000	左移一位
+ 0.1101		+ Y^*
0.1001		余数> 0, 商上1
←1.0010	0.0001	左移一位
+ 1.0011		+ $[-Y^*]_{补}$
0.0101		余数> 0, 商上1
←0.1010	0.0011	左移一位
+ 1.0011		+ $[-Y^*]_{补}$
1.1101		余数< 0, 商上0
←1.1010	0.0110	左移一位
+ 0.1101		+ Y^*
0.0111		余数> 0, 商上1
	← 0.1101	商左移一位

2、补码加减交替除法

补码加减交替除法算法规则

- (1) 操作数均为补码表示，两数的符号位参加运算；
  商符由运算自动产生；
- (2) 除法运算前先检测0：
  只要X、Y有任意一个为0，不进行实际运算；
  若X为0，则商为0；
  若Y为0，按非法除数处理；
- (3) 运算前应满足条件：X*<Y*，否则按溢出处理；
- (4) 设商的初值为0（被除数单倍字长）或为被除数低位部分（被除数双倍字长）；
  部分余数的初值为被除数（被除数单倍字长）或为被除数的高位部分（被除数双倍字长）；
- (5) 为防止运算过程中产生暂时性的溢出，部分余数可采用单符号位或双符号位运算，余数也是补码形式；

❒ (6) 部分余数运算：

试减：{ X、Y同号，做减法：$[X]_{补}+[-Y]_{补}$
X、Y异号，做加法：$[X]_{补}+[Y]_{补}$

上商：{ 余数与Y同号，商上1，余数和商左移一位，做减法；
余数与Y异号，商上0，余数和商左移一位，做加法；}

重复n步

❒ (7) 第n+1步：余数不移位，商单独左移一位恒置1；

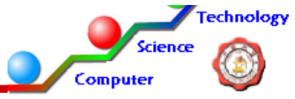
若不需要求余数，除法结束；

若要求求余数：{ 余数与X同号，除法结束。
余数与X异号，应恢复余数；

恢复余数方法：{ 若余数与Y同号，做减法。
若余数与Y异号，做加法。

懒得看了，交给期末了

补码加减交替除法举例



X = - 0.1001
Y = +0.1010
求： X÷Y=?
解： $[X]_{补}$ =1.0111
$[Y]_{补}$ =0.1010
$[-Y]_{补}$=1.0110

$[X÷Y]_{补}$= 1.0001
$[R]_{补}$= 1.1100×2^{-4}
X÷Y= - 0.1111

被除数（余数）	商	说　明
1 1. 0 1 1 1	0. 0 0 0 0	X与Y异号
+ 0 0. 1 0 1 0		$+[Y]_{补}$
0 0. 0 0 0 1		余数与Y同号，商上1
← 0 0. 0 0 1 0	0. 0 0 0 1	左移一位
+ 1 1. 0 1 1 0		$+[-Y]_{补}$
1 1. 1 0 0 0		余数与Y异号，商上0
← 1 1. 0 0 0 0	0. 0 0 1 0	左移一位
+ 0 0. 1 0 1 0		$+[Y]_{补}$
1 1. 1 0 1 0		余数与Y异号，商上0
← 1 1. 0 1 0 0	0. 0 1 0 0	左移一位
+ 0 0. 1 0 1 0		$+[Y]_{补}$
1 1. 1 1 1 0		余数与Y异号，商上0
← 1 1. 1 1 0 0	0. 1 0 0 0	左移一位
+ 0 0. 1 0 1 0		$+[Y]_{补}$ *(如不求余数可省)*
0 0. 0 1 1 0	← 1. 0 0 0 1	*商左移一位末位恒置1*
+ 1 1. 0 1 1 0		$+[-Y]_{补}$ *(余数与X异号，需恢复余数)*
1 1. 1 1 0 0		

若整数：被除数用双倍字长运算

原码、补码加减交替除法规则对照表

除法类型	符号位参与运算	加减次数	移位		备　注
			方向	次数	
原码加减交替法	否	**n+1或n+2**	左	**n**	若最终余数为负，需恢复余数
补码加减交替法	是	**n+1或n+2**	左	**n**	末位恒置1 若最终余数与被除数异号，需恢复余数

注：n为除数的数值部分的位数

阵列乘除法器

阵列乘法：

阵列乘法器

基本思路

- 利用全加器阵列，完全由硬件直接计算乘法结果
- 以 5 位无符号整数为例

$$
\begin{array}{cccccccccc}
 & & & & a_4 & a_3 & a_2 & a_1 & a_0 & =A \\
 & & & \times & b_4 & b_3 & b_2 & b_1 & b_0 & =B \\
\hline
 & & & & a_4b_0 & a_3b_0 & a_2b_0 & a_1b_0 & a_0b_0 & \\
 & & & a_4b_1 & a_3b_1 & a_2b_1 & a_1b_1 & a_0b_1 & & \\
 & & a_4b_2 & a_3b_2 & a_2b_2 & a_1b_2 & a_0b_2 & & & \\
 & a_4b_3 & a_3b_3 & a_2b_3 & a_1b_3 & a_0b_3 & & & & \\
+ & a_4b_4 & a_3b_4 & a_2b_4 & a_1b_4 & a_0b_4 & & & & \\
\hline
P_9 & P_8 & P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0 =P
\end{array}
$$

其中 $P_{ij} = a_i \times b_j$ —— 位积

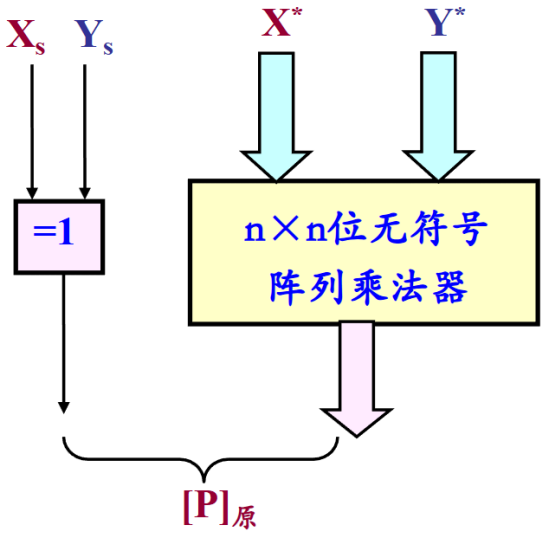
保留进位加法：本级的两数相加，产生的进位信号暂时保留，参加下一级加法器高一位相加的加法。相应的加法器叫做保留进位加法器（CSA）。与普通加法器的区别在全加器进位输入输出端的连接方法上。

原码阵列乘法器

设 被乘数 $[X]_{原} = X_s . X_1X_2 \cdots X_n$

乘　数 $[Y]_{原} = Y_s . Y_1Y_2 \cdots Y_n$

则　积　$[P]_{原}=[X \times Y]_{原} =(X_s \oplus Y_s) . (X^* \times Y^*)$



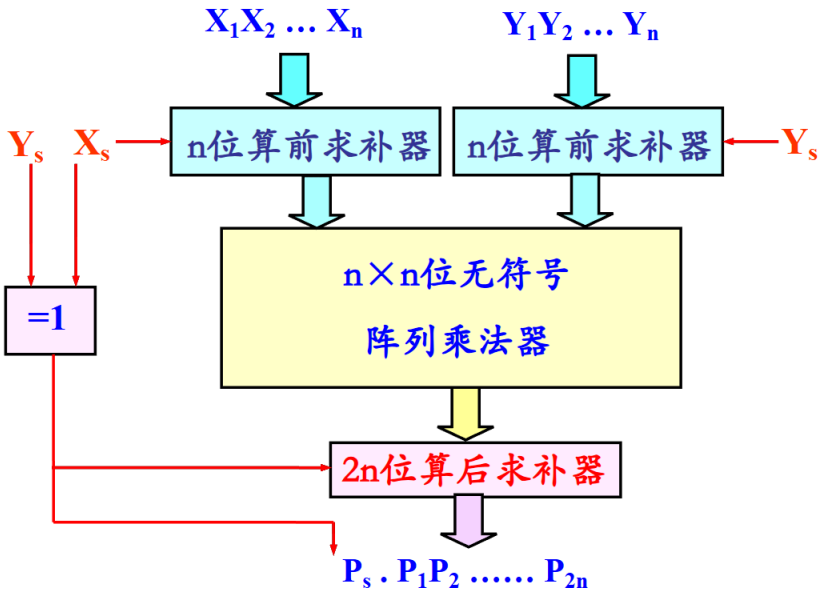
带求补器的补码阵列乘法器

图中给出了带求补器的补码阵列乘法器。更好的方法是利用加/减法阵列电路直接实现Booth算法，完成两个补码的比较相乘过程。限于课时不做讨论。

设 被乘数 $[X]_{补} = x_f . x_1 x_2 \cdots x_n$

乘　数 $[Y]_{补} = y_f . y_1 y_2 \cdots y_n$

则　$[P]_{补} = [X \times Y]_{补} = P_s . P_1 P_2 \cdots P_{2n}$



阵列除法：

❒ 可控加/减法单元CAS

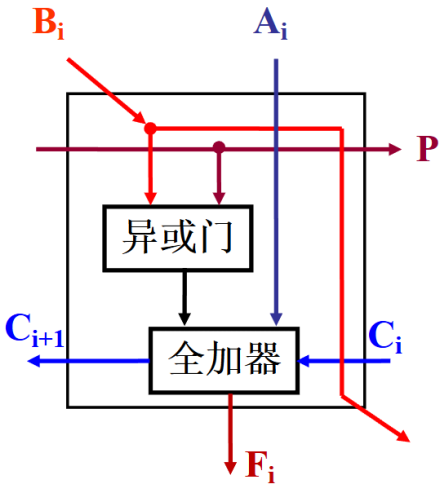
$$\left.\begin{aligned} F_i &= A_i \oplus (B_i \oplus P) \oplus C_i \\ C_{i+1} &= A_i C_i + (A_i + C_i)(B_i \oplus P) \end{aligned}\right\}$$

- **P = 0** 时，输入的操作数不求补，控制做加法

$$\left.\begin{aligned} F_i &= A_i \oplus B_i \oplus C_i \\ C_{i+1} &= A_i B_i + (A_i + B_i) C_i \end{aligned}\right\}$$

- **P = 1** 时，输入的操作数求补，控制做减法

$$\left.\begin{aligned} F_i &= A_i \oplus \overline{B}_i \oplus C_i \\ C_{i+1} &= A_i \overline{B}_i + (A_i + \overline{B}_i) C_i \end{aligned}\right\}$$



可控加减法单元CAS

阵列除法过程：

第一步：试减，P=1，实现 $X^*+[-Y^*]_{补}$ 。

因为X*<Y*, 所以一定不够减，则最高位进位C_{i+1} =0，

利用此进位输出产生商和下一步的P（= C_{i+1}）。

第二步：P=0，做 X^*+Y^* 。

当最高位进位C_{i+1} =1时，表示够减，则 $q_1 = 1$, P =1;

当最高位进位C_{i+1} =0时，表示不够减，则 $q_1 = 0$, P =0。

第三步和第四步：P=0时,做 X^*+Y^*；P=1时,做 $X^*+ [-Y^*]_{补}$ 。

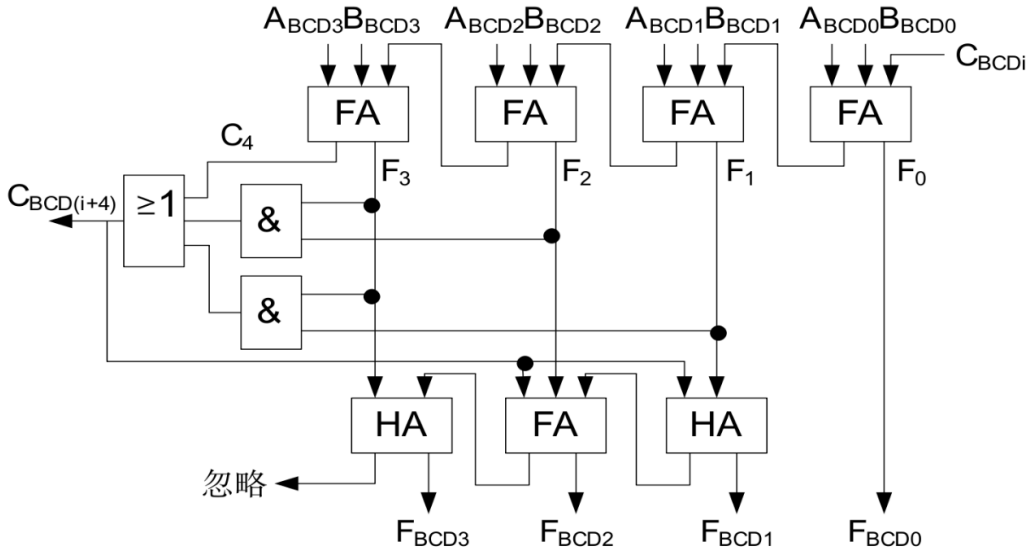
上商和P值产生的规则与第二步相同。

十进制运算

十进制加法：用BCD码（8421）实现，并进行修正

修正规则：C=0，F为0-9（1001），不用修正；其他+6（F+0110），C=1.

十进制数	8421码和 $C_{BCD}\ F_{BCD3}\ F_{BCD2}\ F_{BCD1}\ F_{BCD0}$	校正前的和 $C_4\ F_3\ F_2\ F_1\ F_0$	校正关系
0~9	0 0 0 0 0 …… 0 1 0 0 1	0 0 0 0 0 …… 0 1 0 0 1	不校正
10	1 0 0 0 0	0 1 0 1 0	+6校正
11	1 0 0 0 1	0 1 0 1 1	
12	1 0 0 1 0	0 1 1 0 0	
13	1 0 0 1 1	0 1 1 0 1	
14	1 0 1 0 0	0 1 1 1 0	
15	1 0 1 0 1	0 1 1 1 1	
16	1 0 1 1 0	1 0 0 0 0	
17	1 0 1 1 1	1 0 0 0 1	
18	1 1 0 0 0	1 0 0 1 0	
19	1 1 0 0 1	1 0 0 1 1	

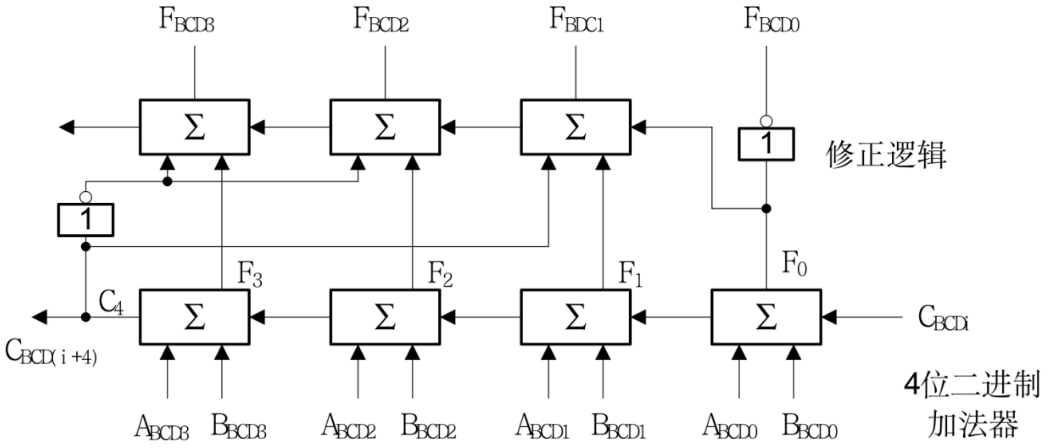


结构特点：由一级四位二进制加法器、一级简化的四位二进制+6加法器和进位产生逻辑三部分组成。

余3码加法修正：C=0，减3（+1101）；C=1，加3（+0011）. C不变。

❖ 余3码加法修正规则：

- ➢ C=0 ， 减3修正。即 $F_{BCD} = F + 1101, C_{BCD} = C$
- ➢ C=1， 加3修正；即 $F_{BCD} = F + 0011, C_{BCD} = C$
- ➢ **一位余3码加法单元逻辑图：**



十进制数	余3码和 C_{BCD} F_{BCD3} F_{BCD2} F_{BCD1} F_{BCD0}	校正前的和 C_4 F_3 F_2 F_1 F_0	校正关系
0	0 0 0 1 1	0 0 1 1 0	-3校正
1	0 0 1 0 0	0 0 1 1 1	
2	0 0 1 0 1	0 1 0 0 0	
3	0 0 1 1 0	0 1 0 0 1	
4	0 0 1 1 1	0 1 0 1 0	
5	0 1 0 0 0	0 1 0 1 1	
6	0 1 0 0 1	0 1 1 0 0	
7	0 1 0 1 0	0 1 1 0 1	
8	0 1 0 1 1	0 1 1 1 0	
9	0 1 1 0 0	0 1 1 1 1	
10	1 0 0 1 1	1 0 0 0 0	+3校正
11	1 0 1 0 0	1 0 0 0 1	
12	1 0 1 0 1	1 0 0 1 0	
13	1 0 1 1 0	1 0 0 1 1	
14	1 0 1 1 1	1 0 1 0 0	
15	1 1 0 0 0	1 0 1 0 1	
16	1 1 0 0 1	1 0 1 1 0	
17	1 1 0 1 0	1 0 1 1 1	
18	1 1 0 1 1	1 1 0 0 0	
19	1 1 1 0 0	1 1 0 0 1	

例题：

8421码加法举例

- 【例6.51】用8421码加法求57+48=?
- 解：$(57)_{BCD}=0101，0111；(48)_{BCD}=0100，1000$

  为清楚起见，这里将各位BCD码之间用逗号隔开。

```
    0101，0111 ——57
+   0100，1000 —— 48
    1001，1111 ——低位无进位
+       0110 —— +6修正
       1，0101 —— 产生出低位进位
    1010，0101 —— 低位进到高位，高位无进位
+   0110       —— +6修正
0001，0000，0101 —— 105
```

余3码加法举例

- 【例6.52】用余3码求316+258=?
- 解：$(316)_{E3}=0110，0100，1001；$

  $(258)_{E3}=0101，1000，1011$

```
  0110，0100，1001     —— 316
+ 0101，1000，1011     —— +258
  1011，1101，0100
+ 1101，1101，0011     —— 有进位+3（+0011），无进位-3（+1101）
  1000，1010，0111     —— 574
```

逻辑运算

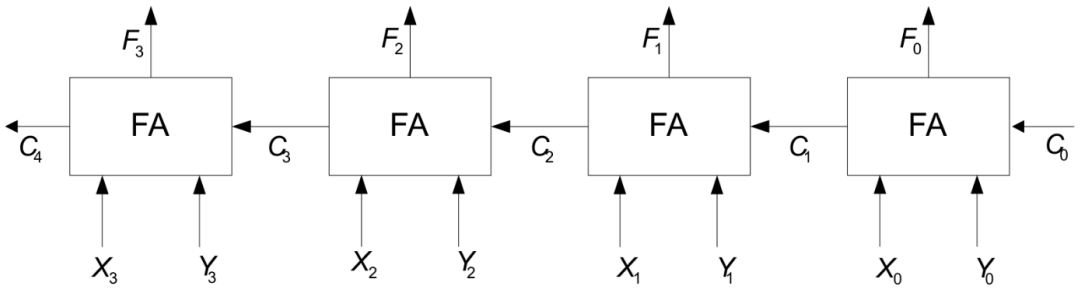
与 或 非 异或

定点运算器的实现

加法器的进位技术

1、行波进位加法器：串行进位的并行加法器，由n个全加器进位输入输出端首尾相连组成，串行进位链。每位进位输出都依赖于低位进位输入。

❒ 四位行波进位加法器逻辑图

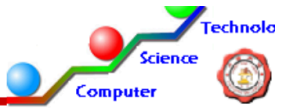


全加器进位：定义进位函数

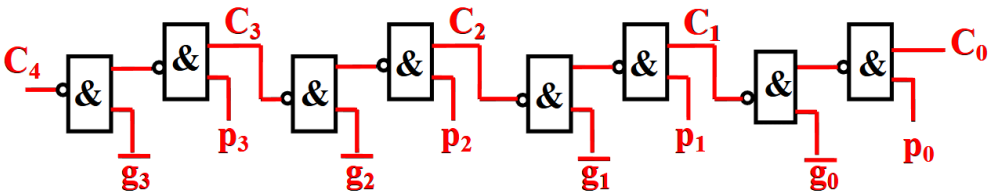
- 进位生成函数：$g_i = X_i Y_i$
- 进位传递函数：$p_i = X_i \oplus Y_i$

进位$C_{i+1} = g_i + p_i C_i$

串行进位链(行波进位线路)



❒ 若采用与非逻辑电路，其进位链结构如下：



❒ 速度分析：若设与非门的延迟时间为t_y，4位加法器进位时延则为8ty，n位加法器最长进位时间为$2nt_y$。n=16时，最长进位时间=$32t_y$。

❒特点：进位延迟时间长，影响加法器速度。

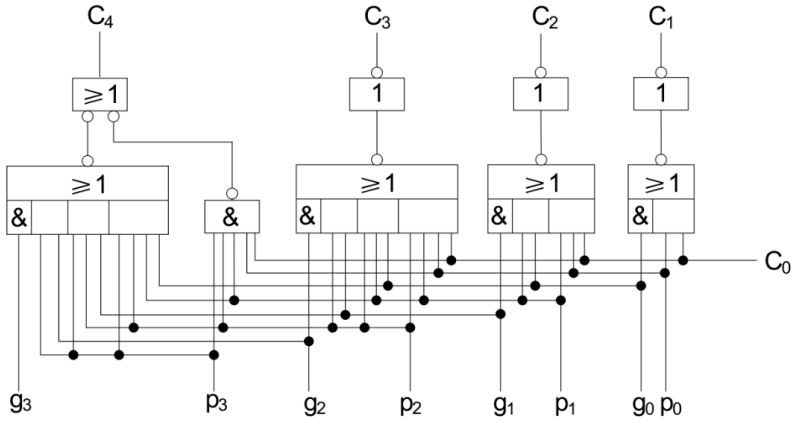
2、先行进位加法器（并行）：不依赖前一位的进位结果。

❒ 仍以四位加法器为例，将前一位进位信号表达式直接代入本位，得到一组新的进位表达式：

$$
\begin{aligned}
C_1 &= g_0 + p_0 C_0 \\
C_2 &= g_1 + p_1 g_0 + p_1 p_0 C_0 \\
C_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0 \\
C_4 &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_0
\end{aligned}
$$

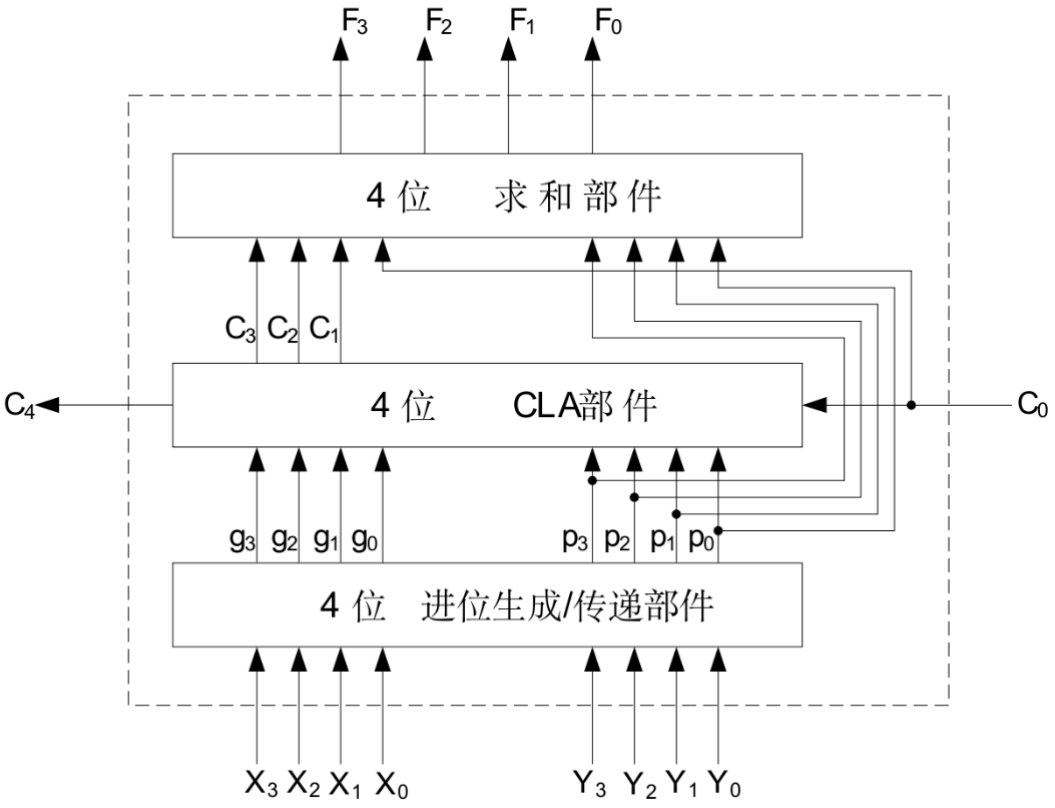
❒ 这组表达式中各位进位信号直接由进位函数和最低位进位输入产生，而进位函数是直接由加数生成的，与低一位进位信号无关。

❒ 由与或非-与非逻辑实现的四位先行进位线路（CLA部件）如图所示：



❒ 速度分析：假设与或非门时延为1.5ty，与非门时延仍为1ty，忽略产生进位函数的时延，则产生进位输出共需两级门2.5ty左右的时延。这个延迟时间是常数，各位进位都一样，与加法器的位数无关。

一个完整的先行进位加法器可看成由**进位函数产生线路（进位生成/传递部件）、求和线路（求和部件）、先行进位线路（CLA部件）**三部分构成。

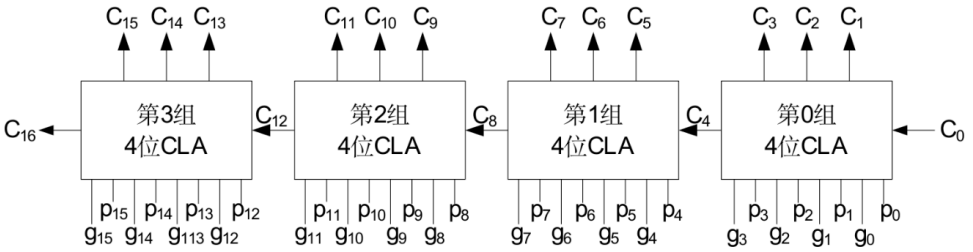


限制：由于门电路扇入系数（最多输入端数量）的影响，先行进位加法器的规模一般最多只能到8位左右

改良：

1. 成组先行-级联进位：组内先行进位，组间串行进位

❒ 16位成组先行—级联进位框图



❒ **特点：二级进位结构，组内为第一级，用4位CLA部件实现；组间构成第二级，用串行进位方式实现。**

❒ **速度分析：组内先行进位线路时延为2.5ty，16位4分组的最长进位时延达2.5ty×4=10ty。比16位行波进位最长时延32ty快了2倍多，是一种较好的进位加速折衷方案。**

位数较多时加速作用下降。

2. 多级先行进位：分组，组内、组间都先行进位

❒ **16位多级先行进位加法器举例：仍采用4-4-4-4分组方案，小组进位信号C_{n+4}不再由组内的先行进位线路产生，而是由组间先行进位线路产生。组间进位信号的通式为：**

$$C_{n+4}= g_{n+3}+p_{n+3}g_{n+2}+p_{n+3}p_{n+2}g_{n+1}+p_{n+3}p_{n+2}p_{n+1}g_n+p_{n+3}p_{n+2}p_{n+1}p_nC_n$$

❒ **式中前4项为本组产生进位的条件，第5项则反映低一组进位信号通过本组传递到高一组的条件。可以定义两个小组进位函数：**

$$G_i = g_{n+3}+p_{n+3}g_{n+2}+p_{n+3}p_{n+2}g_{n+1}+p_{n+3}p_{n+2}p_{n+1}g_n$$

小组进位生成函数

$$P_i = p_{n+3}p_{n+2}p_{n+1}p_n$$

小组进位传递函数

❒ **函数的下标 i 表示组号，在16位加法器中，i=0（n=0），1（n=4），2（n=8），3（n=12）。**

❒ **将小组进位函数代入小组进位信号表达式中可得：**

$$\left.\begin{aligned} C_4 &= G_0 + P_0 C_0 \\ C_8 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\ C_{12} &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\ C_{16} &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \end{aligned}\right\}$$

❒ **由这组表达式可以看出，组间先行进位的逻辑结构与组内先行进位线路完全相同，其输入都依赖于进位函数和最低位进位，输出均为进位信号。因此，我们可以使用通用的先行进位部件CLA来构成组间的先行进位线路。**

❒ **通常，最低级进位函数p_i、g_i称为一级进位函数；小组进位函数P_i、G_i称为二级进位函数；三级进位函数在本书中通常用P^*、G^*表示；……**

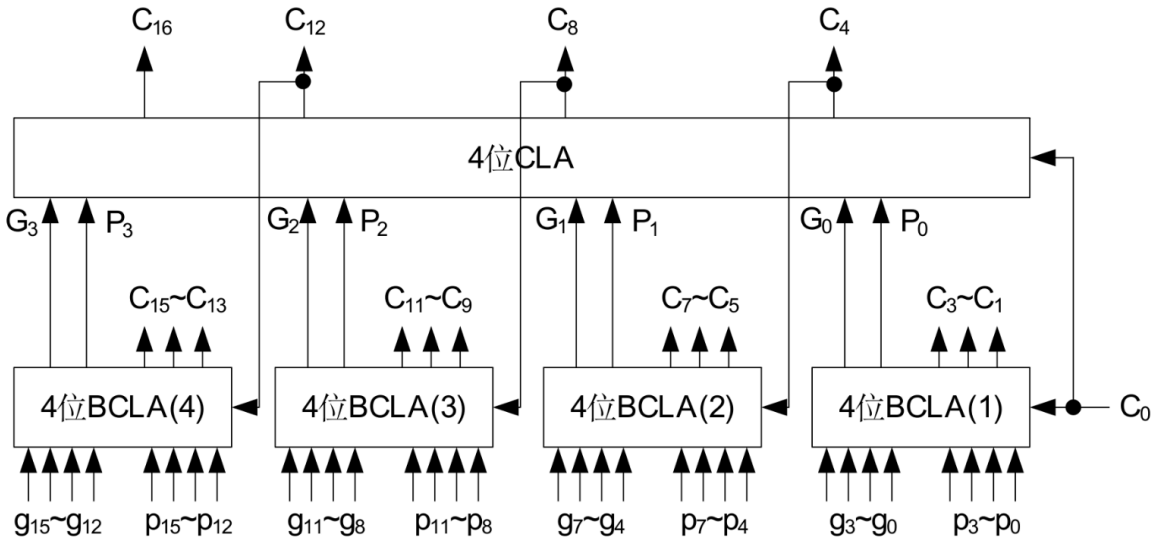
3、运用成组先行进位部件（BCLA）：输出组间进位函数P、G。

4位BCLA部件的输入输出逻辑

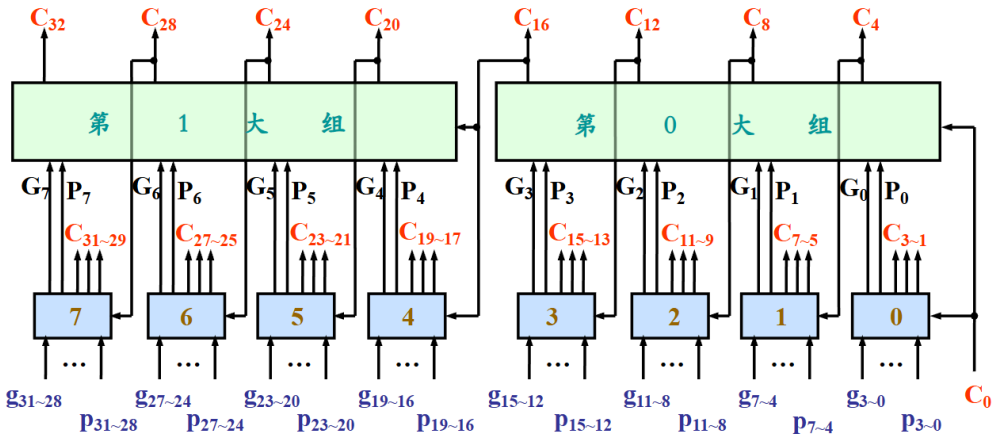
$$
\begin{aligned}
C_{n+1} &= g_n + p_n C_n \\
C_{n+2} &= g_{n+1} + p_{n+1} g_n + p_{n+1} p_n C_n \\
C_{n+3} &= g_{n+2} + p_{n+2} g_{n+1} + p_{n+2} p_{n+1} g_n + p_{n+2} p_{n+1} p_n C_n \\
G_n &= g_{n+3} + p_{n+3} g_{n+2} + p_{n+3} p_{n+2} g_{n+1} + p_{n+3} p_{n+2} p_{n+1} g_n \\
P_n &= p_{n+3} p_{n+2} p_{n+1} p_n
\end{aligned}
$$

式中n仍然表示不同组的下标增量，C_{n+1}~C_{n+3}为组内进位输出；p_n~p_{n+3}、 g_n~g_{n+3}为组内的进位函数输入；P_n、 G_n为组间进位函数输出。

4-4-4-4分组的16位二级先行进位加法器逻辑框图



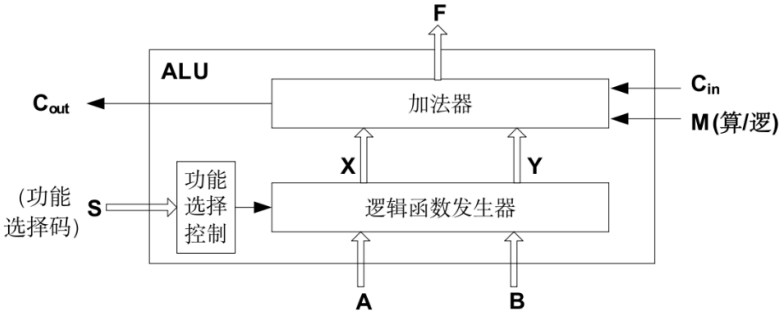
32位并行加法器二级先行-级联进位链框图



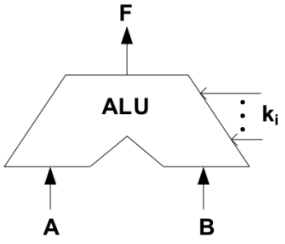
共四个阶段，每个阶段2.5t_y

算术逻辑单元ALU

ALU：同时兼有算术、逻辑运算功能并具备简单的选择控制能力的运算电路，是运算器核心部件，由加法器、函数发生器、功能选择电路三部分组成

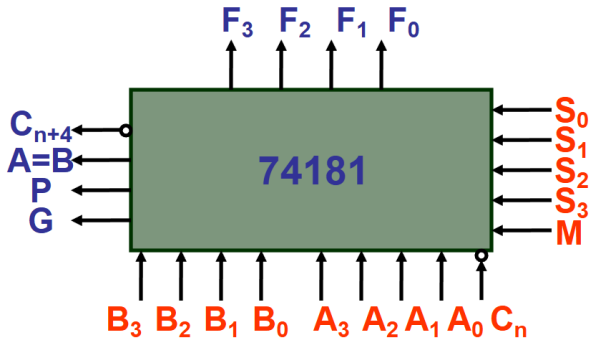


（a）内部结构

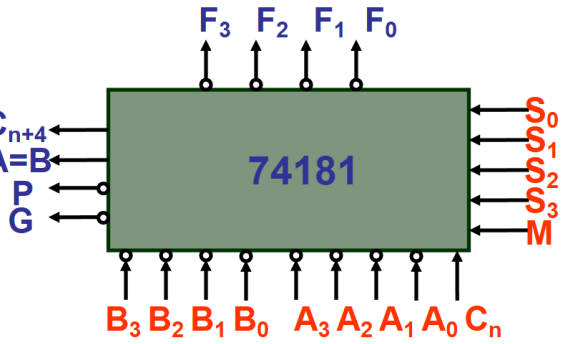


（b）逻辑符号

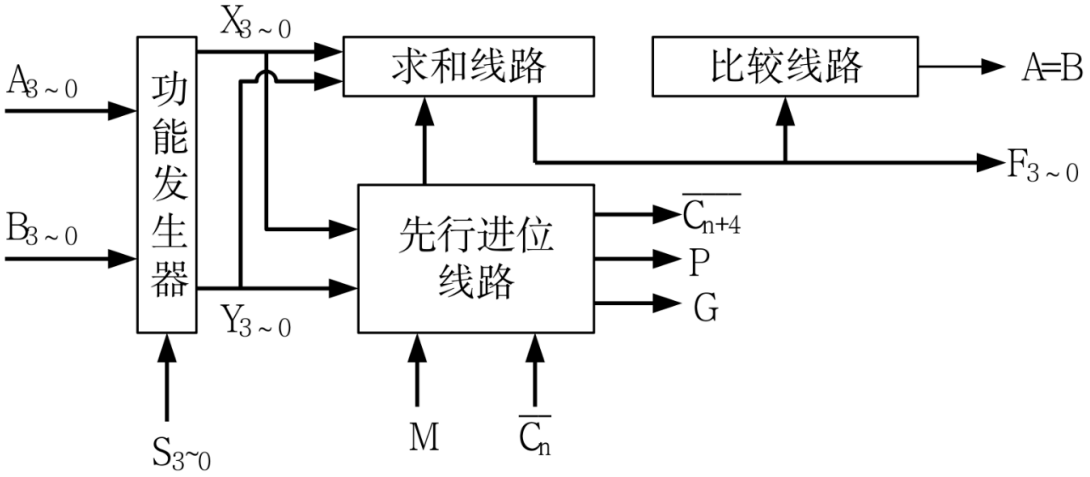
多功能算术逻辑运算单元芯片74181：4位二进制，16种功能，支持正负。由逻辑函数发生器（功能发生器）、求和线路、先行进位线路、比较线路四部分组成。



正逻辑



负逻辑



期末看一下：

- **逻辑函数发生器。**对原始输入数据A_3~A_0、B_3~B_0进行逻辑组合，产生出X_3~X_0、Y_3~Y_0两路逻辑函数输出。

  - 第 i 位函数发生器的逻辑表达式

$$X_i = \overline{S_3A_iB_i + S_2\overline{A_iB_i}}$$

$$Y_i = \overline{A_i + S_0B_i + S_1\overline{B_i}}$$

（正逻辑）

  - 设计特点：当进行加法运算时，X_i、Y_i既是加数，又是进位函数g_i、p_i，即：

$$g_i = X_i \wedge Y_i = Y_i$$

$$p_i = X_i \vee Y_i = X_i$$

- **求和电路**

两级异或门组成，以正逻辑为例其等效逻辑表达式为：

$$F_i=X_i\oplus Y_i\oplus (C_{n+i}+M)$$

当M=H时，选择逻辑运算：$F_i= X_i\oplus Y_i\oplus 1=X_i\odot Y_i$

当M=L时，选择算术运算：$F_i= X_i\oplus Y_i\oplus C_{n+i}$

- **先行进位电路**

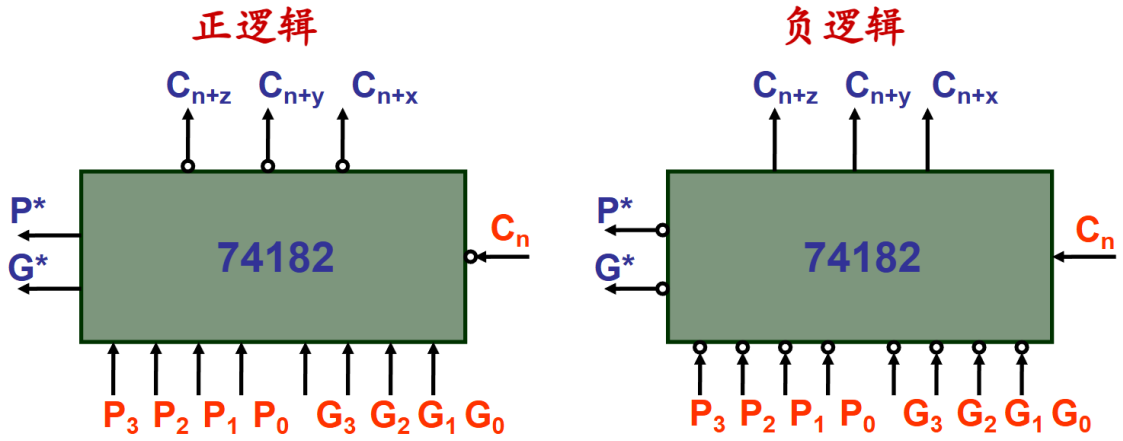
兼有CLA、BCLA两种先行进位功能，四位CLA电路产生C_{n+4}~C_{n+1}四个进位信号，C_{n+3}~C_{n+1}为组内进位不输出，小组间进位信号C_{n+4}通过相应的引脚输出。74181还可输出小组进位函数P、G，供外接的BCLA芯片使用。

- **比较电路**

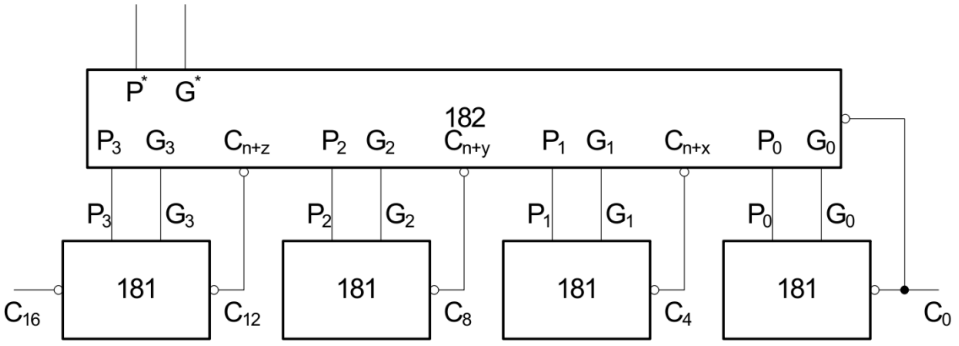
74181内部设置了一个比较门，当F_3~F_0为全0时，其输出端A=B输出有效的高电平比较信号。

多片74181进位输入输出串联组成多位成组先行-级联进位ALU。

74182是与74181配套的通用4位成组先行进位部件。为了减少延迟时间，进位信号采用了与进位函数输入反相的输出方式。



❒ 4片74181与1片74182联合可构成16位二级先行进位的ALU，如下图（正逻辑为例）：



❒ 特点

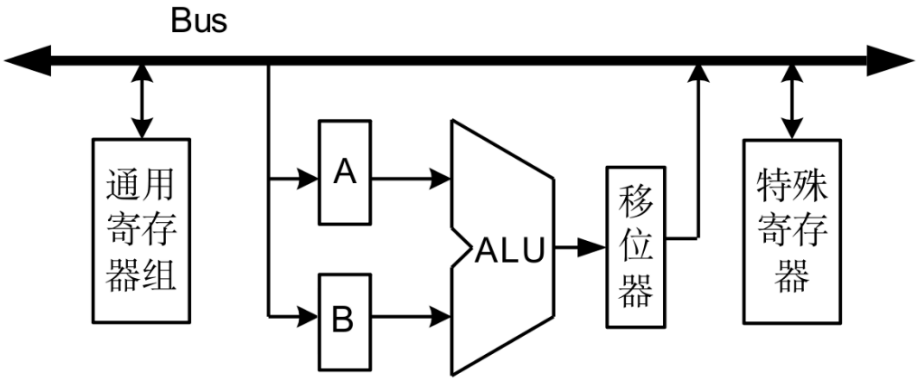
16位ALU仍分4组，组内先行进位由74181内部实现，但组间不再使用74181的进位输出端C_{n+4}，而是将74181输出的小组进位函数P、G信号送往74182，再通过74182产生组间先行进位信号。

定点运算器的基本结构

运算器：完成对数据的算术/逻辑运算（ALU），暂存参加运算的原始数据和中间结果（通用寄存器组），设置乘商寄存器（专用寄存器）及移位器，连接通过多路选择器完成，内部数据传送通过内部总线完成。运算器常称为CPU的数据通路。

❒ 分散互连结构的运算器

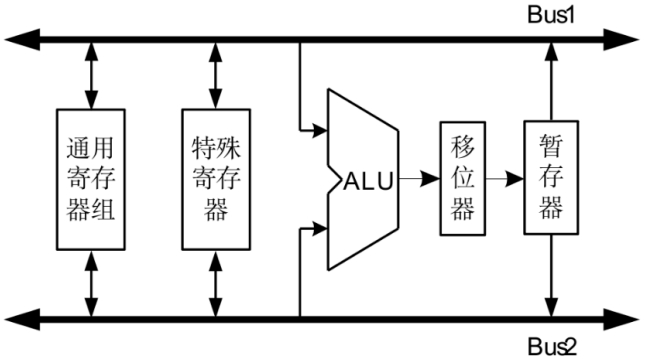
❒ 单总线结构的运算器



❒ 特点

- 需要设置2个暂存器协调ALU输入/输出端同时传送问题；
- 完成一次加法运算需要3个CPU时钟周期的时间；
- 结构简单、规整，运算速度慢。

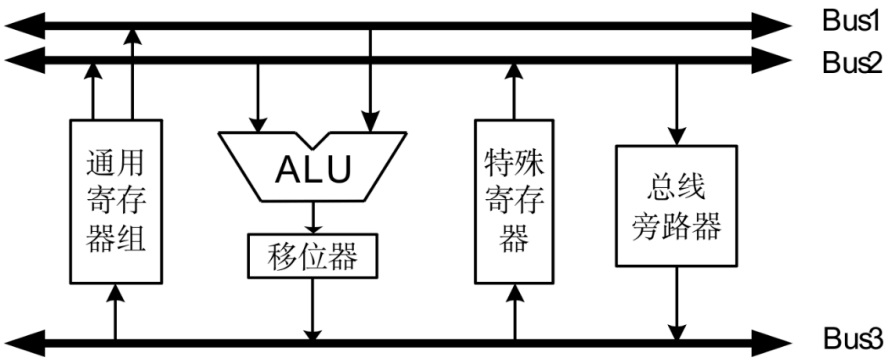
❒ 双总线结构的运算器



❒ 特点

- ALU的输入/输出端只需要设一个暂存器。
- 运算器完成一次加法运算需要2个CPU时钟周期的时间。
- 两条总线之间的传输可通过ALU进行，或通过设置专用的总线连接器实现。

❒ 三总线结构的运算器



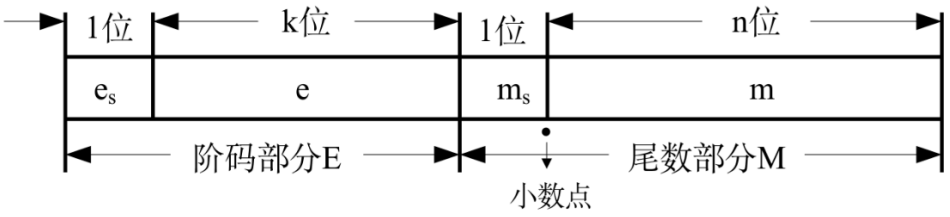
❒ 特点

❍ 运算器完成一次加法运算只需要1个CPU时钟周期的时间，运算速度不再受总线分时使用的限制，在此总线已不是影响运算器性能的主要因素。

浮点数的表示与运算

浮点表示法：数值×指数（$N = M \times R^E$）. M称为尾数，用小数表示；E称为阶码，用整数表示；R称作阶的基数或底。

❒ 浮点数的一般格式



❒ 浮点机器码书写形式

E_s，E_k…E_2E_1；M_s . $M_1M_2M_3$…$M_{n-1}M_n$

❒ 浮点数阶码的位数决定浮点数的表示范围；

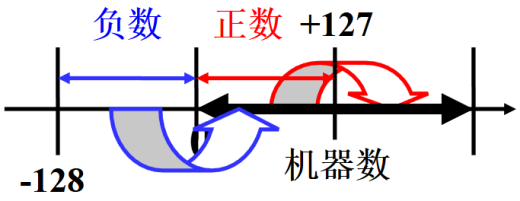
尾数的位数决定浮点数的有效精度。

移码：一位符号位和k位数值位组成，$[E_{移}] = 2^k + E(-2^k \le E < 2^k)$，$2^k$称为移码的偏置常数/位移量。几何意义：将k位数真值在数轴上向右平移2^k个位置。

例如：1位符号位和 7 位数值位组成的移码, 其定义为：

$[E]_{移} = 2^7 + E \quad -2^7 \leq E < 2^7$

编码集合：00000000～11111111



8 位移码表示的机器数为：

数的真值在数轴上向右平移了 128 个位置。

通常移码只用于表示整数，且只用于表示浮点数的阶码部分。

移码和补码的关系：符号位取值相反，数值部分取值相同。

与补码类似，移码零的表示唯一（1000 0000）。

8位机器整数代表不同码制时的真值范围

二进制代码	无符号数对应的真值	原码对应的真值	反码对应的真值	补码对应的真值	移码对应的真值
0000 0000	0	+0	+0	0	−128
0000 0001	1	+1	+1	+1	−127
0000 0010	2	+2	+2	+2	−126
……	……	……	……	……	……
0111 1110	126	+126	+126	+126	−2
0111 1111	127	+127	+127	+127	−1
1000 0000	128	−0	−127	−128	0
1000 0001	129	−1	−126	−127	+1
1000 0010	130	−2	−125	−126	+2
……	……	……	……	……	……
1111 1101	253	−125	−2	−3	+125
1111 1110	254	−126	−1	−2	+126
1111 1111	255	−127	−0	−1	+127

移码范围：$2^{-k} \sim 2^k - 1$

规格化：尾数最高位必须是一个有效数值，$\frac{1}{R} \leq |M| < 1$(补码 ≤ 1)

❒ 当R=2时规格化浮点数尾数形式为

$[M]_{原}$ = 0.1xx…x 或1.1xx…x，判断条件：$M_{MSB}=1$

$[M]_{补}$ = 0.1xx…x 或1.0xx…x，判断条件：$M_s \oplus M_{MSB}=1$

规格化处理：若浮点数超出规格化表示范围，需同时调整尾数和阶码的大小，转化为规格化数。

- 右规：尾数右移1位，阶码+1. 只需进行一次
- 左规：尾数每左移一位，阶码-1，直到符合规格化数条件。

机器0的标准形式：尾数为0，阶码为最小值。

规定浮点数阶码用移码，尾数用补码或原码，通常R=2，阶码范围：$-2^k \sim 2^k - 1$

❒ 当基为2，阶码用移码、尾数用原码表示时（阶移尾原格式），规格化浮点数的取值范围为：

最小负数：$-2^{2^k-1} \times (1-2^{-n})$

最大负数：$-2^{-2^k} \times 2^{-1}$

} 负数可表示范围

机 器 零：0

最小正数：$2^{-2^k} \times 2^{-1}$

最大正数：$2^{2^k-1} \times (1-2^{-n})$

} 正数可表示范围

补码负数：1.00000000 - 1.01111111 $\rightarrow -1 \sim -2^{-1} + 2^{-n}$

❒ 当基为2，采用阶移尾补格式表示时，规格化浮点数的取值范围则变为：

最小负数：-2^{2^k-1}

最大负数：$-2^{-2^k} \times (2^{-1}+2^{-n})$

} 负数可表示范围

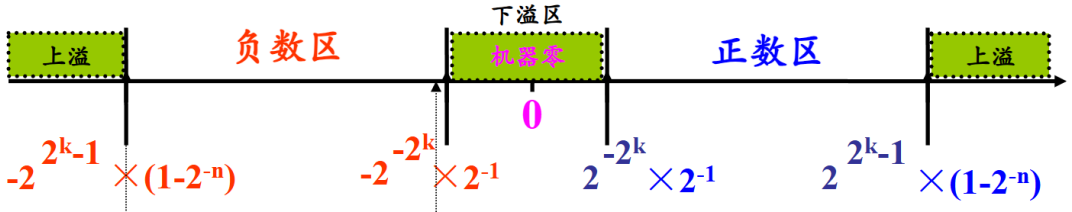
机 器 零：0

最小正数：$2^{-2^k} \times 2^{-1}$

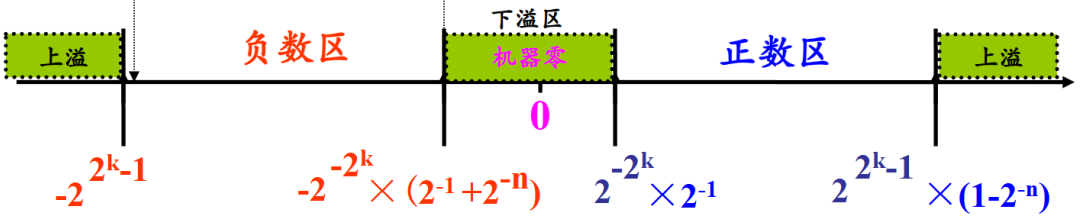
最大正数：$2^{2^k-1} \times (1-2^{-n})$

} 正数可表示范围

阶码用移码，基为2；尾数用原码



阶码用补码，基为2；尾数用补码



❒ 浮点数表示范围由正数域、机器0和负数域三部分组成。
❒ 真0是数轴上一个点，机器0对应数轴上0点附近的一段区间

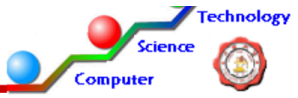
浮点数溢出：阶码超出其表示范围

- 上溢：运算结果大于浮点数可表示的最大阶。分为正负上溢。正上溢属于溢出出错。
- 下溢：运算结果小于最小阶。分为正负下溢。下溢时作为机器零。
- 若用补码或移码表示阶码，设置双符号位判断溢出。

尾数采用原码规格化表示时，其绝对值的最高数值位必须为1，为提高表数精度，可采用隐藏位技术。

基数变大，浮点数表示范围明显增大，表示精度下降，在数轴上排列更稀疏。

定点表示法与浮点表示法的比较



(1) 浮点数比定点数表示范围大；

(2) 浮点数有效精度比同字长定点数低。浮点表示法实质是牺牲精度为代价来扩大数据表示范围；

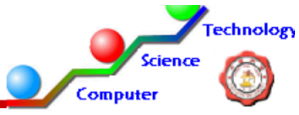
(3) 溢出判断方法不同。浮点运算以阶码上溢为溢出标志、定点数则以数值是否超出范围破坏了符号位为溢出标志；

(4) 浮点数由阶码和尾数两部分组成。比定点数结构复杂，运算步骤增多，运算速度下降，运算线路复杂；

(5) 由于表示范围大，浮点数运算时能有效减少运算结果产生溢出的次数，减轻编程时选择比例因子的工作量。

浮点数在表示范围、溢出处理和编程的方便性上优于定点数，而在表示精度、运算速度及运算的复杂性等方面又不如定点数。

浮点表示法对性能结构的影响



按照对浮点功能的设置方式，通常可以将计算机分为：

(1) **定点机：**只能表示和处理定点数，指令系统不支持任何浮点功能。结构简单，成本低廉。

(2) **定点机+浮点选件：**浮点协处理器是一种专门用来对计算机中的浮点数进行处理的LSI芯片，其内部主要包括浮点运算部件和浮点处理部件，可在定点机原有指令系统的基础上执行一套附加的浮点指令，完成对定点机浮点功能的扩展。协处理器无法单独工作，只能和主CPU协同运行。浮点协处理器选件使浮点运算的速度大大提高。

(3) **浮点机：**具有浮点运算指令和浮点运算器，实数处理速度和处理能力都十分强大。

- 要求用最少的位数设计一个浮点数格式，必须满足下列要求：

（1）十进制数的范围：负数 $-10^{38}\sim-10^{-38}$；

正数 $+10^{-38}\sim+10^{38}$。

（2）精度：7位十进制数据。

- 解：

（1）浮点数表示范围的设计主要是确定阶码的位数。

采用试探法：　由$2^{10}>10^3$

可得　$(2^{10})^{12}>(10^3)^{12}$，　即$2^{120}>10^{36}$

又因　$2^7>10^2$

所以　$2^7\times2^{120}>10^2\times10^{36}$，　即$2^{127}>10^{38}$

同理可得　$2^{-127}<10^{-38}$

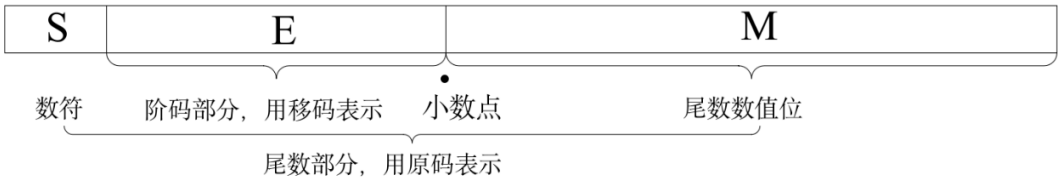
- 由上分析可知：阶码取8位（含1位阶符），且用补码或移码表示时，对应的浮点数取值范围为 $2^{-128}\sim2^{+127}$，可以满足题意要求。

（2）浮点数精度的设计主要是确定尾数的位数。

由于$10^7\approx2^{23}$，故尾数的数值部分可取23位。加上1位数符位，可达24位。

- 最终该浮点数至少取32位，其中阶码8位（含1位阶符），尾数24位（含1位数符），基数R=2。

IEEE754浮点标准：



- 为了便于讨论，我们把IEEE 标准浮点数格式书写为

$S,\ E_sE_k\cdots\cdots E_2E_1;\ .M_1M_2\cdots\cdots M_n$

类型	数符S	阶码E	隐藏位	尾数值M	总位数	偏置常数	
						十六进制	十进制
短实数	1	8	有	23	32	7FH	+127
长实数	1	11	有	52	64	3FFH	+1023
临时实数	1	15	无	64	80	3FFFH	+16383

S(31)	E(30-23)	M(22-0)

754标准格式规定：

- 由数符S、阶码E、尾数M三部分组成，指数以2为底、尾数以2为基数；
- 符号位S占1位，安排在最高位，S=0表示正数，S=1表示负数；
- 阶码E占8位，移码表示，偏移量（偏置常数）为+127；
- 尾数M占低23位，用原码表示，小数点在尾数域的最前面；
- 尾数域表示的值是1.M。由于最高有效位总是1，可将1隐藏在小数点左边不予存储，尾数实际24位。则一个32位浮点数的实际真值为：

  $X=(-1)^S\times(1.M)\times 2^{E-127}$ (E:1~254)
- 当阶码E=0…0且尾数M=0…0时，表示的真值X=±0；
- 当阶码E=1…1且尾数M=0…0时，表示的真值X=±∞；
- 当阶码E=0…0且尾数M不为0时，若隐藏位=0，表示非规格化数。
- 当阶码E=1…1且尾数M不为0时，用来表示非数（即不是一个数）。

舍入：

- 就近舍入：舍入成最接近的可表示数，类似0舍1入。
- 朝0舍入：截断法
- 朝+∞舍入：向上舍入。若尾数为正，只要需舍去的多余位不全为0，做最低有效位+1；若尾数为负，则截断
- 朝−∞舍入：向下舍入。与前面相反。

IEEE浮点数特点：

1. 符号位放最高位，便于判零和判断符号
2. 机器0为一串0
3. 比较时不用区分阶码和尾数，直接比较
4. 隐藏位为1
5. 移码采用2^k-1为偏置常数

1、将$(100.25)_{10}$转换成短浮点数格式。

解：(1) 把十进制数转换成二进制数

$(100.25)_{10}=(110\ 0100.01)_2$

(2) 规格化二进制数

$110\ 0100.01=1.1001\ 0001\times2^{110}$

(3) 计算出阶码的移码（偏置值＋阶码真值）

111 1111(127H)＋110 ＝1000 0101

(4) 以短浮点数格式存储该数

该数的符号位＝0，阶码＝1000 0101

尾数＝.100 1000 1000 0000 0000 0000 （23位）

所以，(100.25)10的短浮点数代码为：

0，1000 0101；.100 1000 1000 0000 0000 0000

十六进制值为：42C8 8000H

2、将短浮点数 C1C9 0000H 转换成十进制数。

解：（1）把十六进制数转换成二进制形式，并分离出符号位、阶码和尾数

因为，C1C9 0000H＝1 1000 0011 100 1001 0000 0000 0000 0000B

所以，　符号位＝1　　（1位）

阶码＝1000 0011　　（8位）

尾数＝.100 1001 0000 0000 0000 0000　　（23位）

（2）计算出阶码的真值（即移码－偏置值）

1000 0011 -111 1111＝100

（3）以规格化二进制数形式写出此数：$1.1001\ 001\times2^{100}$

（4）写成非规格化二进制数形式：1 1001.001

（5）转换成十进制数，并加上符号位

$(-1\ 1001.001)_2=(-25.125)_{10}$

规格化浮点加减运算：小数点对齐，考虑阶码、尾数两部分，受规格化表示范围限制

步骤：对阶（小数点对齐，小阶向大阶对齐）、尾数加减（定点规则）、规格化、舍入（右移时舍入）、结果判溢出

❒ 设有两个浮点数X和Y，分别表示为

$$X = M_x \times R^{E_x} \qquad Y = M_y \times R^{E_y}$$

❒ 若求：Z=X±Y=?

❒ 则浮点加减运算的基本规则可用通式进行描述（*R*=2）

$$Z = X \pm Y = \begin{cases} (M_x \pm M_y \times 2^{-(Ex-Ey)}) \times 2^{Ex}, & E_x \geq E_y \\ (M_x \times 2^{-(Ey-Ex)} \pm M_y) \times 2^{Ey}, & E_x < E_y \end{cases}$$

其中：$2^{-(Ex-Ey)}$和$2^{-(Ey-Ex)}$称为移位因子，反映出对阶时尾数右移的位数。对阶的规则为“小阶向大阶看齐”。

方法:

❒ 0操作数检测

运算前先分别对两操作数X、Y进行判0操作

若X=0，则令X+Y=Y；X-Y=-Y；运算结束；

若Y=0，则令X+Y= X-Y=X；运算结束。

❒ 对阶

阶码的比较通过“求阶差”操作完成 。

$\Delta E = E_x - E_y$

❍ 若$\Delta E=0$，$E_x=E_y$，尾数可直接加减；

❍ 若$\Delta E\neq 0$，$E_x\neq E_y$，需要对阶。

✧若$\Delta E>0$，$E_x>E_y$，E_y+1，$M_y\rightarrow 1$，$\Delta E-1$

重复到$\Delta E=0$为止。此时$E_z=E_x$

✧若$\Delta E<0$，$E_x<E_y$，E_x+1，$M_x\rightarrow 1$，$\Delta E+1$

重复到$\Delta E=0$为止。此时$E_z=E_y$

❒ 尾数相加减

- ❍ 按定点小数加减运算规则进行

$$M_z=M_x \pm M_y$$

- ❍ 当尾数用补码表示时有：

$$[M_z]_{补} = [M_x \pm M_y]_{补} = \begin{cases} [M_x]_{补} + [M_y]_{补} & \text{求和} \\ [M_x]_{补} + [-M_y]_{补} & \text{求差} \end{cases}$$

❒ 结果规格化

浮点加减运算结果可能发生两种非规格化情况：

- ❍ 尾数溢出，右规；
- ❍ 尾数非0，但不满足规格化条件。左规。随着尾数逐位左移，可以将保护位逐位补入尾数的最低位。

❒ 舍入

规格化处理后，可能还剩余有若干保护位需要舍去。常用的舍入方法可根据具体情况选用。在此约定本节的舍入处理均采用“0舍1入”法。

❒ 溢出判断

以阶码是否溢出作为判断条件。阶码上溢时作为溢出出错，转溢出处理；阶码下溢时表示为机器零即可。

浮点加减运算有可能导致阶码上溢的操作是右规和舍入。

- ❍ 由于右规过程中尾数右移一位时阶码+1，而如果阶码已为最大值，再加1就会向上超出其表示范围。
- ❍ 在采用“0舍1入”法进行舍入时，如果尾数末位加1后溢出，会引起右规，……接下来就和右规时上溢的情形一样了。

> 溢出判断操作不是等浮点加减运算结束时再做，而是在运算过程中需要的步骤立即进行。

例题：

- 已知：$X= 2^{010} \times 0.11011011$，

  $Y= 2^{100} \times (-0.10101100)$

  采用阶补尾补格式表示，

  求：$Z_1=X+Y=?$　　$Z_2=X-Y=?$

- 解：首先，写出X、Y的浮点机器数表示：

  阶码用 5 位补码（含2位符号位）　　尾数用 10 位补码（含2位符号位）

  $[X]_{阶补尾补} = 00, 010;\ 00.11011011$

  $[Y]_{阶补尾补} = 00, 100;\ 11.01010100$

$[X]_{阶补尾补} = 00, 010;\ 00.11011011$

$[Y]_{阶补尾补} = 00, 100;\ 11.01010100$

（1）对阶：

$[\Delta E]_{补} = [E_X]_{补} + [-E_Y]_{补} = 00, 010 + 11, 100 = 11, 110$

$\Delta E = -2 < 0$，　即 $E_X < E_Y$

因此，$[M_X]_{补} = 00.00110110(11)$（即右移2位）

$[E_x]_{补} = 00, 100$

$[X]_{阶补尾补} = 00, 100;\ 00.00110110(11)$

（2）尾数求和差

加：		减：	
	00. 00110110(11)		00. 00110110(11)
+	11. 01010100	+	00. 10101100
	11. 10001010(11)		00. 11100010(11)

$[M_{z1}]_{补}$=11. 10001010(11)； $[M_{z2}]_{补}$=00. 11100010(11)

（3）结果规格化

本例和应左规一次操作，故得

和：$[Z_1]_{阶补尾补}$ = 00, 011； 11. 00010101(1)

差：$[Z_2]_{阶补尾补}$ = 00, 100； 00. 11100010(11)

（4）舍入

本例采用 0 舍 1 入方案得

$[Z_1]_{阶补尾补}$ = 00, 011； 11. 00010101， 舍

$[Z_2]_{阶补尾补}$ = 00, 100； 00. 11100011， 入

（5）判溢出

本例中，和、差的阶码均不溢出。

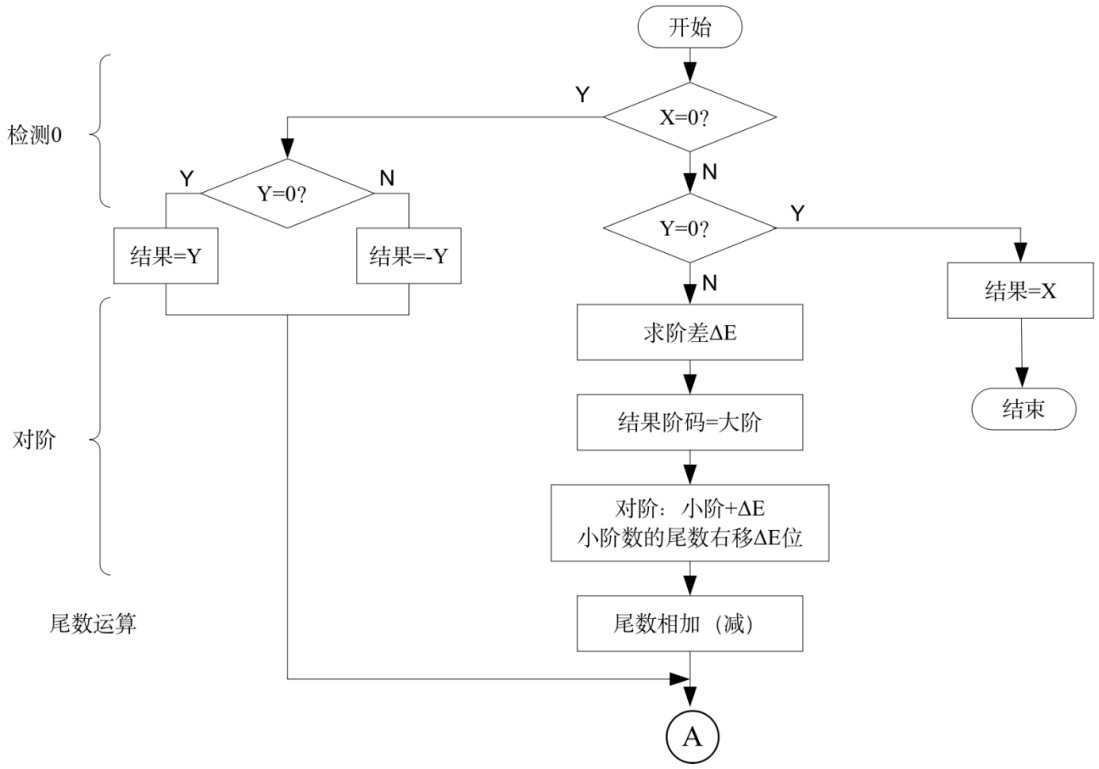
$[Z_1]_{阶补尾补}$= 00, 011； 11. 00010101

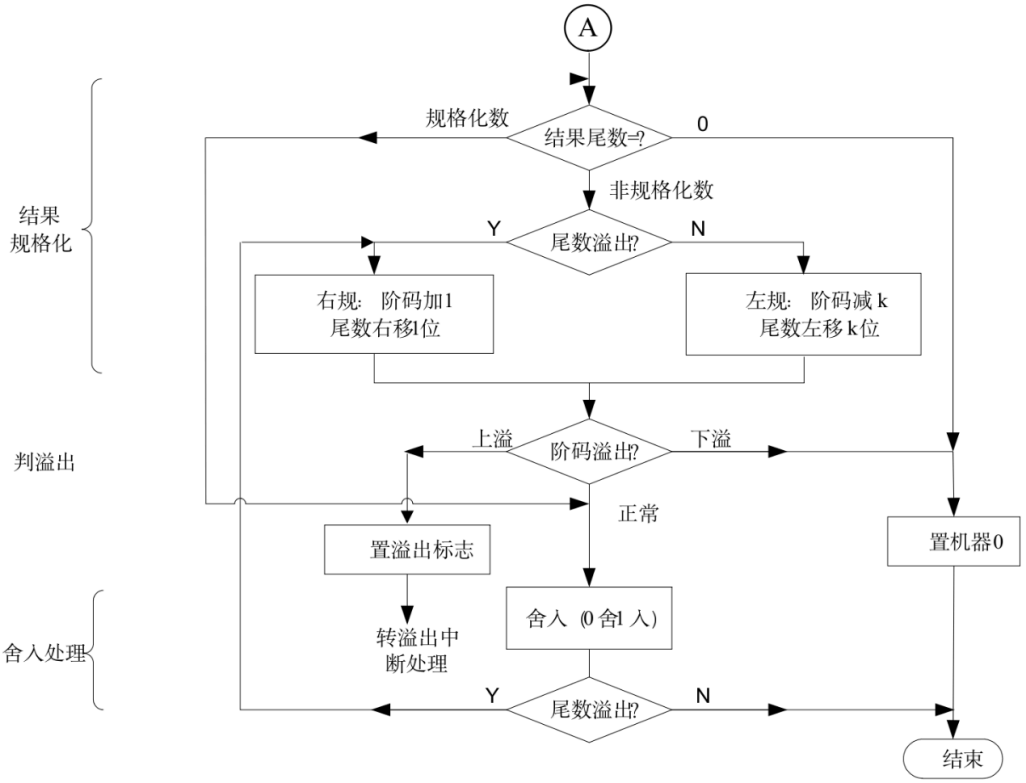
$[Z_2]_{浮阶补尾}$= 00, 100； 00. 11100011

最后得：$Z_1 = 2^{011} \times (-0.11101011)$

$Z_2 = 2^{100} \times (0.11100011)$

浮点加减运算操作流程





规格化浮点乘除运算：阶码加减，尾数乘除

(0) 检测0：

乘：X或Y=0，则 X×Y =0，不再运算；

除：X=0，则 X÷Y =0；Y=0，转非法除数处理。

(1) 阶码相加、减：

乘：E_X+E_Y ；

除：$E_X- E_Y$（可能溢出）

(2) 尾数相乘、除：

乘：$M_X\times M_Y$ ；

除：$M_X \div M_Y$

(3) 规格化处理（左规或右规，可能溢出）；

(4) 舍入操作（可能带来又一次右规，可能溢出）；

(5) 判断结果的正确性，即检查阶码是否溢出。

- 其中，(3)、(4)、(5)三步操作方法同浮点加减运算。

移码加减：前移加后补。或相加时-2^k，相减时$+2^k$. 为方便判断溢出，用双符号位，且最高位为0，即正01，负00. 11为下溢，按机器0处理；10为上溢，溢出出错。

- 已知：$X=2^{010}\times 0.1011$，$Y=2^{100}\times(-0.1101)$

  求：$Z=X\times Y=?$

- 解：写出X、Y的浮点机器数表示：

  阶码用 5 位移码（含2位符号位）　　尾数用 5 位补码（含1位符号位）

  $[X]_{阶移尾补}=01,010;\ 0.1011$

  $[Y]_{阶移尾补}=01,100;\ 1.0011$

(1) 阶码相加

$$[E_Z]_{移}=[E_x]_{移}+[E_y]_{补}=01,010+00,100=01,110$$

无溢出

$[X]_{阶移尾补} = 01, 010;\ 0.1011$

$[Y]_{阶移尾补} = 01, 100;\ 1.0011$

（2）尾数相乘

$[M_Z]_{补} = [M_x]_{补} \times [M_y]_{补}$

$= 000.1011 \times 11.00110$

$= 111.01110001$

部分积	乘数 $My_{n-1}My_nMy_{n+1}$
000.0000	11.00 11 0
+111.0101	+$[-M_x]_{补}$
111.0101	
2→111.1101	0111.00 1
+000.1011	+$[M_x]_{补}$
000.1000	
2→000.0010	0001 11. 0
+111.0101	+$[-M_x]_{补}$
111.0111	0001 00 0
	清0

（3）结果规格化

方法与加减法相同。

本例不需要规格化。

（4）舍入处理

方法与加减法相同。

本例采用0舍1入法，需舍。得

$[M_Z]_{补} = 1.0111$

$[E_Z]_{移} = 01, 110$

$[M_Z]_{补} = 111.01110001$

（5）检查溢出

$[E_Z]_{移} = 01, 110$

移码最高符号位为0，本例不溢出。最后得

$[Z]_{阶移尾补} = 01, 110;\ 1.0111$

即 $Z = X \times Y = 2^{110} \times (-0.1001)$

- 已知：$X = 2^{-010} \times 0.1011$，$Y = 2^{100} \times (-0.1101)$

  求：$Q = X \div Y = ?$

- 解：阶码用5位移码（含2位符号位）　尾数用5位原码（含1位符号位）

  则X、Y的机器数形式为

  $[X]_{阶移尾原} = 00, 110;\ 0.1011$

  $[Y]_{阶移尾原} = 01, 100;\ 1.1101$

（1）阶码相减

$[E_q]_{移} = [E_x]_{移} + [-E_y]_{补} = 00, 110 + 11, 100 = 00, 010$

无溢出

$[M_x]_{原} = 0.1011$

$[M_y]_{原} = 1.1101$

（2）尾数相除

$Q_s = (X_s \oplus Y_s)$

$= 0 \oplus 1 = 1$

$M_q^* = M_x^* \div M_y^*$

$= 0.1101$

则

$[M_q]_{原} = 1.1101$

注：也可以多求几位商，以供舍入。

被除数（余数）	商	说　明
0.1011	0.0000	
+ 1.0011		$+[-Y^*]_{补}$（减除数）
1.1011		余数< 0，商上0
← 1.0110	0.0000	左移一位
+ 0.1101		$+Y^*$
0.1001		余数> 0，商上1
← 1.0010	0.0001	左移一位
+ 1.0011		$+[-Y^*]_{补}$
0.0101		余数> 0，商上1
← 0.1010	0.0011	左移一位
+ 1.0011		$+[-Y^*]_{补}$
1.1101		余数< 0，商上0
← 1.1010	0.0110	左移一位
+ 0.1101		$+Y^*$
0.0111		余数> 0，商上1
	← 0.1101	商左移一位

$[Q]_{阶移尾原} = 00，010；\ 1.1101$

（3）结果规格化：已是规格化数；

（4）舍入处理：不必舍入；

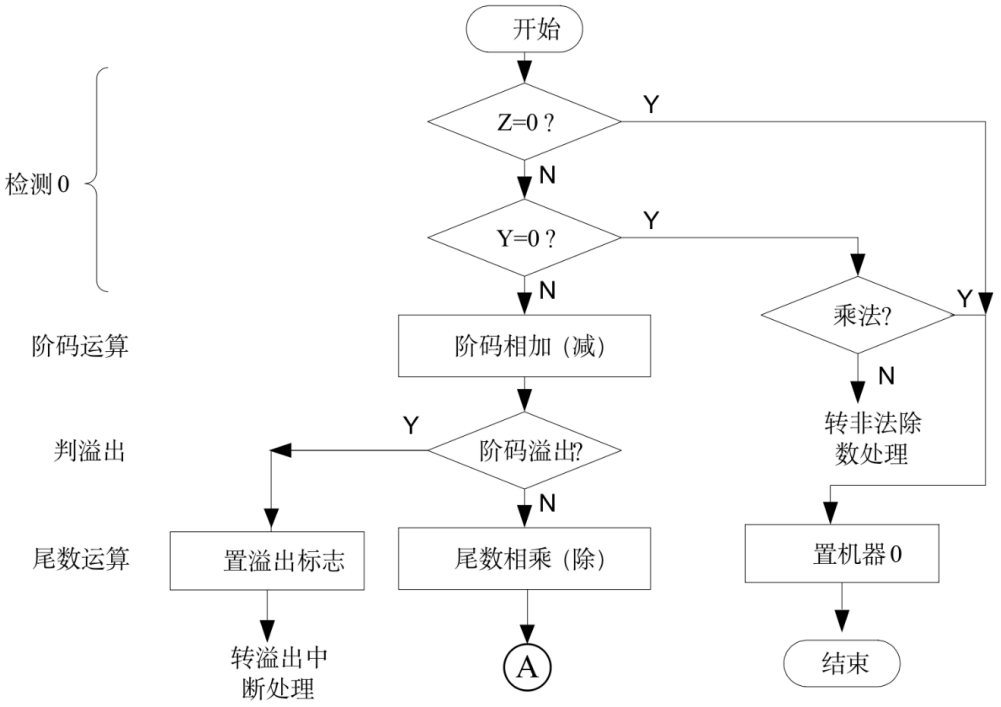
（5）检查溢出：不溢出。

则最终的商

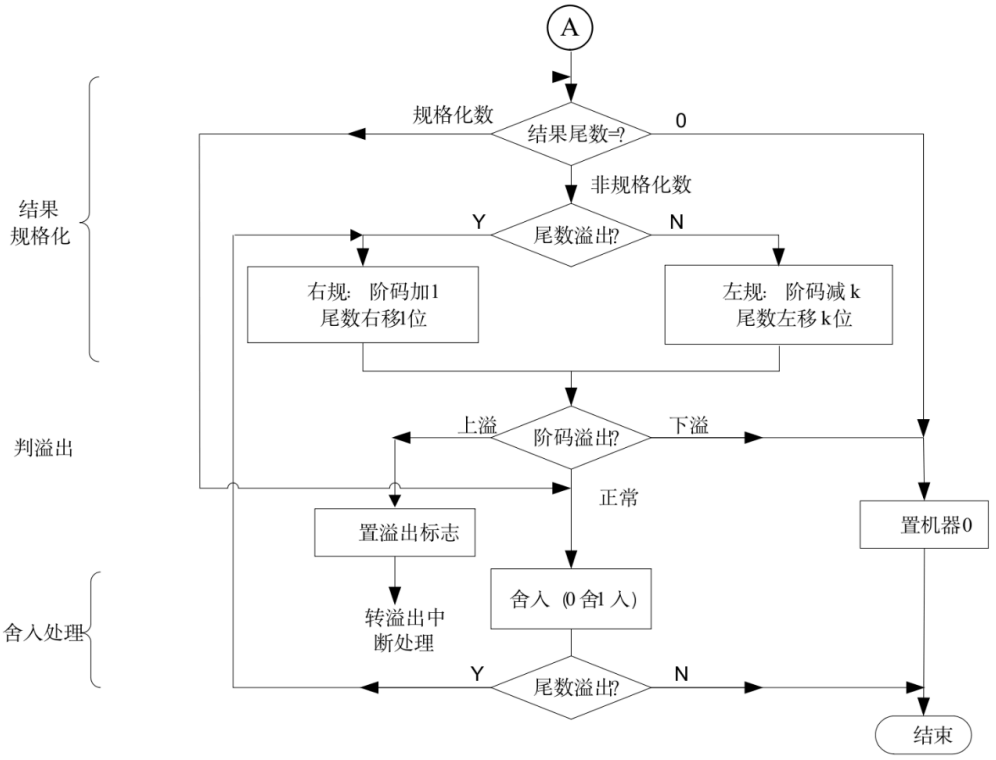
$[Q]_{阶移尾原} = 00, 010;\ 1.1101$

即 $X \div Y = 2^{-110} \times (-0.1101)$

浮点乘除运算操作流程



浮点加减运算操作流程（续）



考点

应掌握的主要内容：　数的机内表示　运算方法　先行进位技术

1. 数据在计算机内的表示

① 二进制数的原、反、补、移码表示

② 浮点数的机内表示

③ 各类机内数据的表示范围

2. 数值数据算术运算的实现算法

① 定点数的算术运算
- 补码加、减法运算
- 原码一位乘法
- 补码一位乘法、二位乘法
- 原码加减交替除法

② 浮点数的算术运算 { 规格化浮点数加、减法运算；规格化浮点数乘、除法运算

③ 移位运算：算术移位、逻辑移位；舍入方法

3. 逻辑运算

4. 先行进位技术

① 行波进位加法器

② 先行进位加法器

③ 成组先行-级联进位

④ 多级先行进位

⑤ 进位延迟时间计算